

DEEP LEARNING FOR COMPUTER VISION

Deep Learning for Computer Vision

- convolutional neural networks (convnets)
- data augmentation
- pretrained convnet
- fine-tuning กับ pretrained convnet
- visualization เพื่อดูว่า convnets เรียนรู้อะไร และตัดสินใจทำ classification อย่างไร

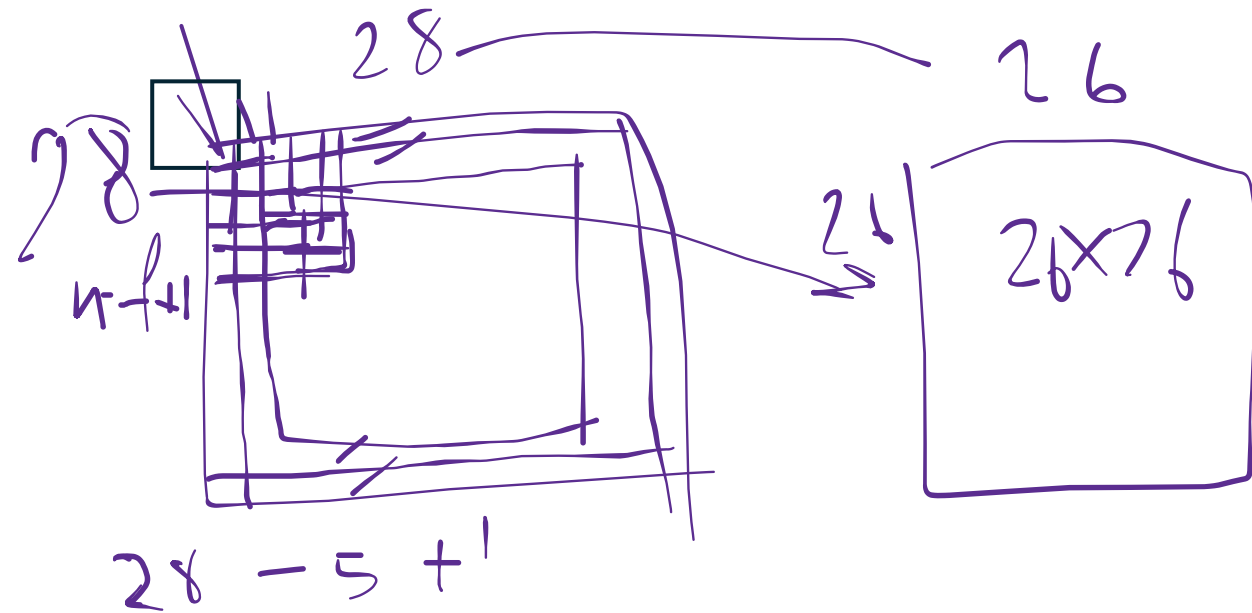
Introduction to Convnets

ConvNet on MNIST

```
from keras import layers
from keras import models
```

```
model = models.Sequential()
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
```

Convnet



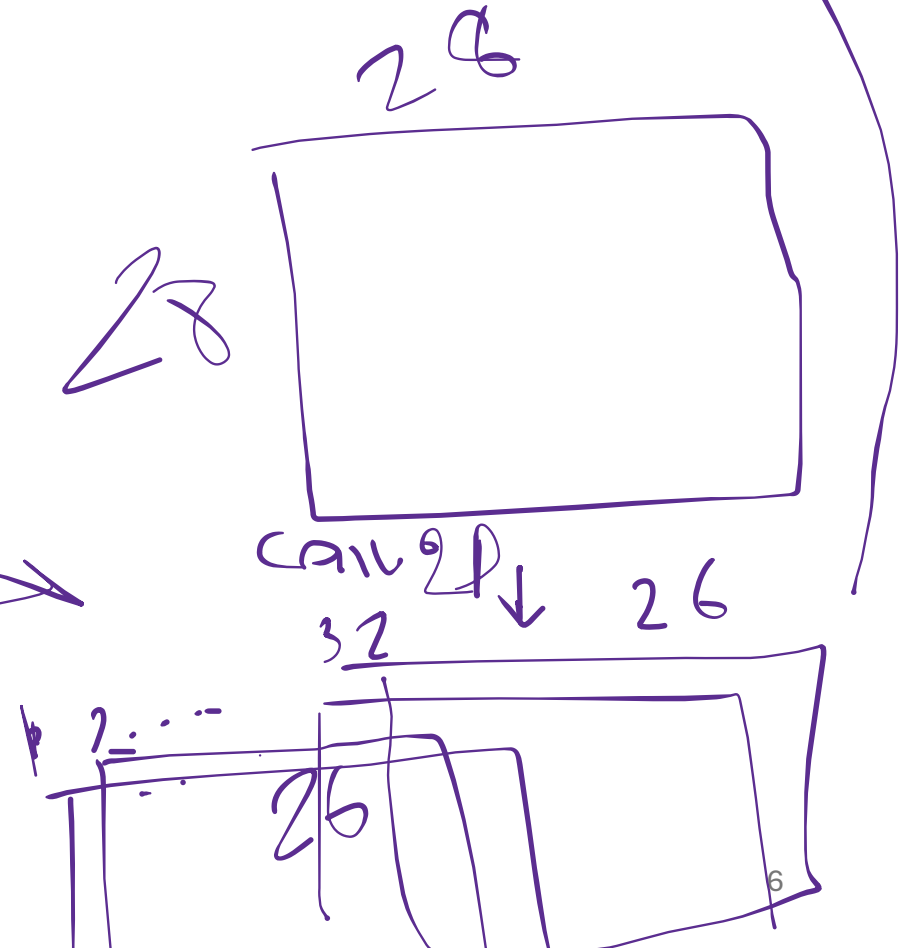
โครงสร้างของ ConvNet พื้นฐาน

- ใช้ Sequential model
- Stack ของ Conv2D
- MaxPooling2D

Input ของ ConvNet

```
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)))
```

- Input shape = (height, width, channels)
- MNIST = (28, 28, 1)



The architecture of the convnets

model.summary()

Layer (type)	Output Shape	Param #
conv2d_1 (Conv2D)	(None, 26, 26, 32)	320
maxpooling2d_1 (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_2 (Conv2D)	(None, 11, 11, 64)	18496
maxpooling2d_2 (MaxPooling2D)	(None, 5, 5, 64)	0
conv2d_3 (Conv2D)	(None, 3, 3, 64)	36928

=====
Total params: 55,744
Trainable params: 55,744
Non-trainable params: 0

Output : Conv2D and MaxPooling2D

- Output = 3D tensor
- (height, width, channels)
- spatial size ลดลงเมื่อ network ลึกขึ้น

Layer (type)	Output Shape
conv2d_1 (Conv2D)	(None, 26, 26, 32)
maxpooling2d_1 (MaxPooling2D)	(None, 13, 13, 32)
conv2d_2 (Conv2D)	(None, 11, 11, 64)
maxpooling2d_2 (MaxPooling2D)	(None, 5, 5, 64)
conv2d_3 (Conv2D)	(None, 3, 3, 64)

28 28 3

$$13 - 3 + 1$$

$$5 - 3 + 1$$

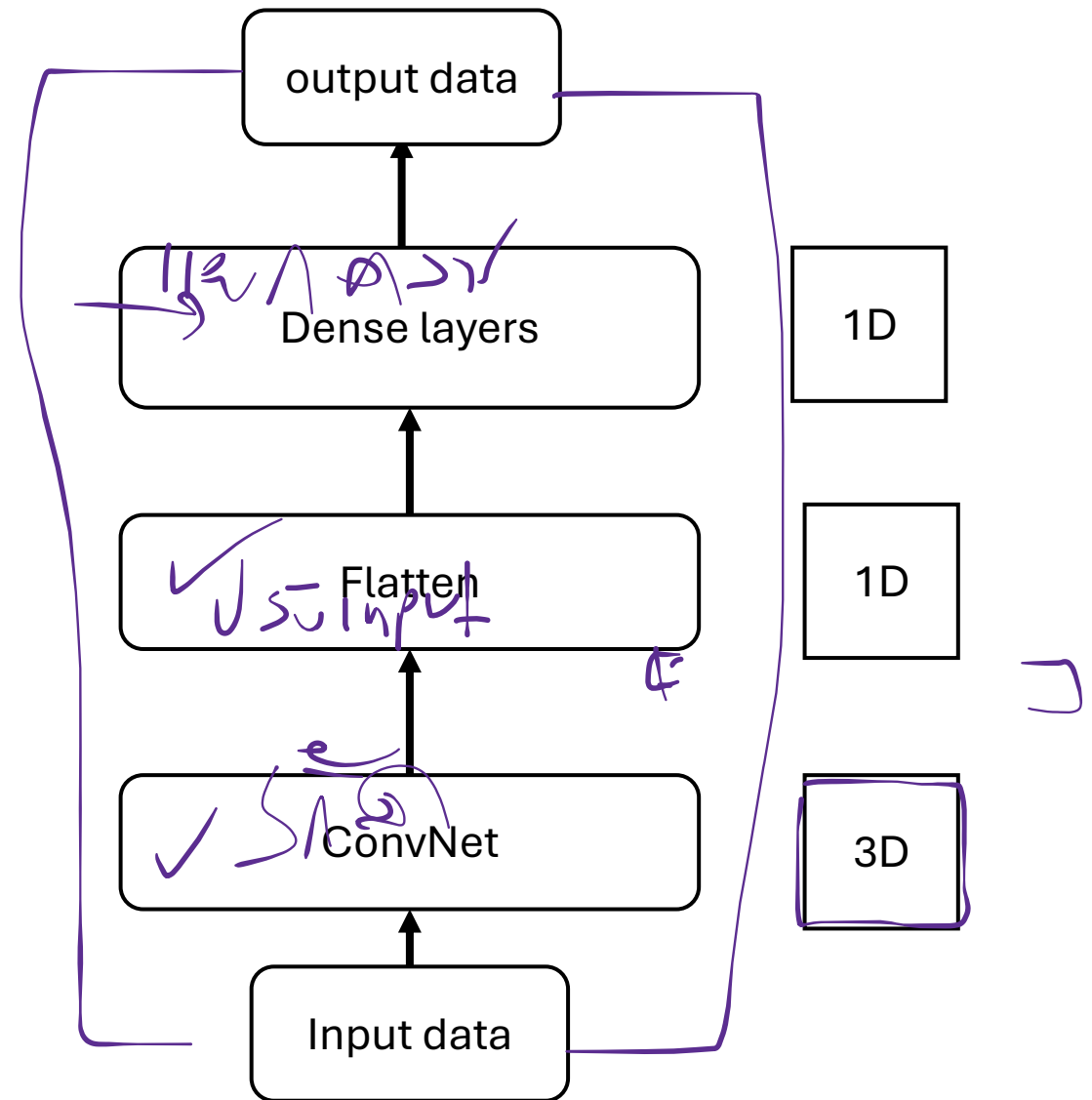
จำนวน Channels

- Conv2D(32, ...) → 32 channels
- Conv2D(64, ...) → 64 channels

Add a classifier

+ Max Pooling

- ConvNet → feature extractor
- Dense layers → classifier



Dense layers

```
model.add(layers.Flatten())  
model.add(layers.Dense(64, activation='relu'))  
model.add(layers.Dense(10, activation='softmax'))
```

- Flatten: 3D $\rightarrow$ 1D
- Dense(64)
- Dense(10, softmax)

The architecture of the convnets + classifier

`model.summary()`

Layer (type)	Output Shape	Param #
conv2d_1 (Conv2D)	(None, 26, 26, 32)	320
maxpooling2d_1 (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_2 (Conv2D)	(None, 11, 11, 64)	18496
maxpooling2d_2 (MaxPooling2D)	(None, 5, 5, 64)	0
conv2d_3 (Conv2D)	(None, 3, 3, 64)	36928
flatten_1 (Flatten)	(None, 576)	0
dense_1 (Dense)	(None, 64)	36928
dense_2 (Dense)	(None, 10)	650

Total params: 93,322

Trainable params: 93,322

Non-trainable params: 0

convnet

input
output

Model on Mnist

```
from keras.datasets import mnist
from keras.utils import to_categorical 1-let

(train_images, train_labels), (test_images, test_labels) = mnist.load_data()

train_images = train_images.reshape((60000, 28, 28, 1))
train_images = train_images.astype('float32') / 255.0

test_images = test_images.reshape((10000, 28, 28, 1))
test_images = test_images.astype('float32') / 255.0

train_labels = to_categorical(train_labels)
test_labels = to_categorical(test_labels)

model.compile(optimizer='rmsprop',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

model.fit(train_images, train_labels, epochs=5, batch_size=64)
```

Evaluation

```
test_loss, test_acc = model.evaluate(test_images, test_labels)
```

```
test_acc
```

```
>>> 0.99080000000000000001
```

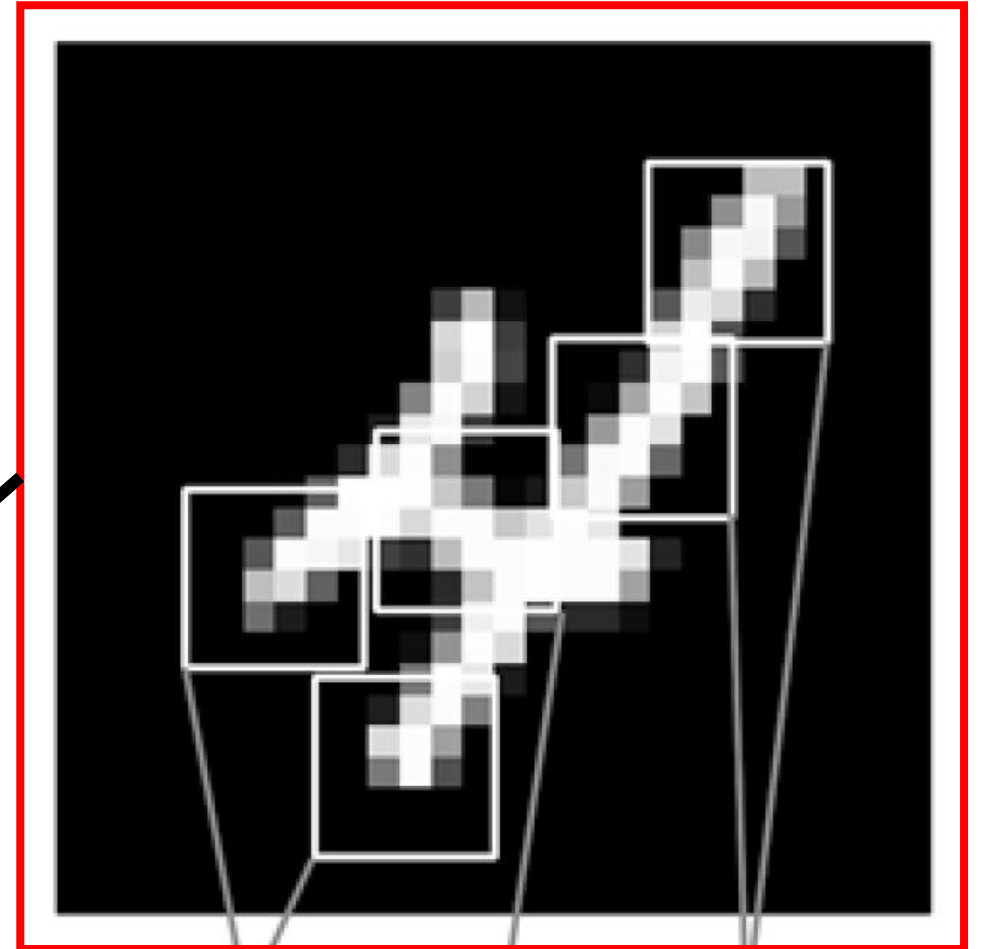
- densely connected network จากบทก่อนให้ประมาณ 97.8%

Introduction to Convnets

Dense vs Convolution

- Dense layer: global patterns
- Convolution layer: local patterns

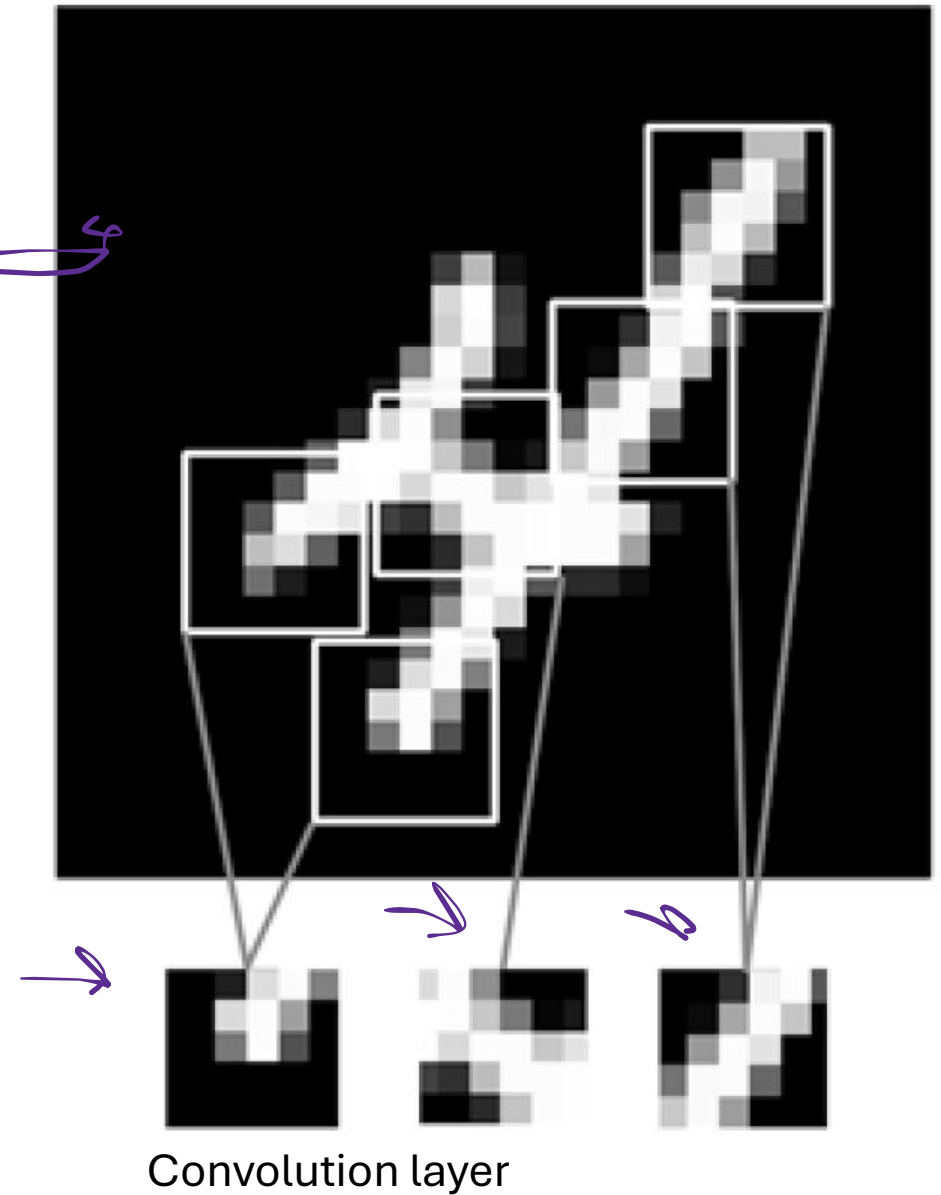
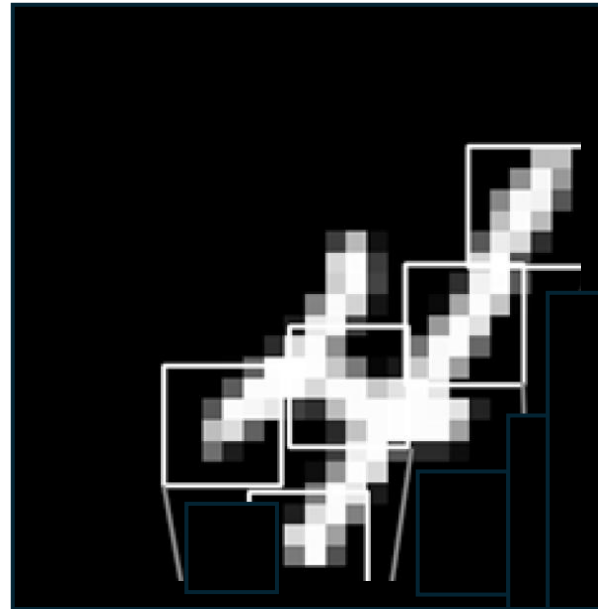
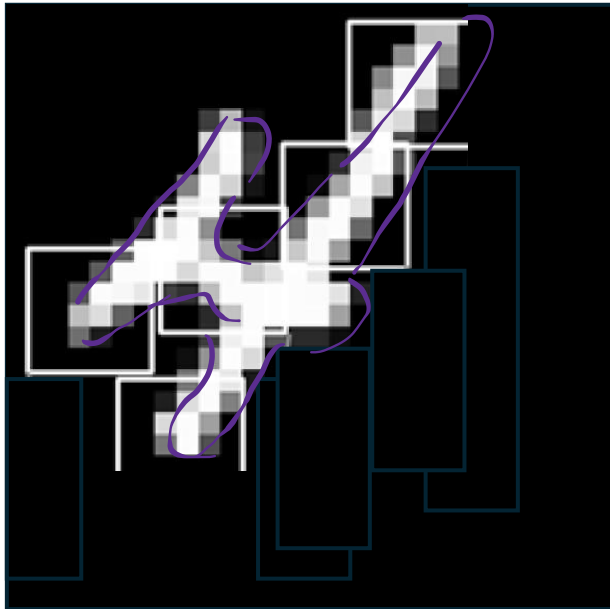
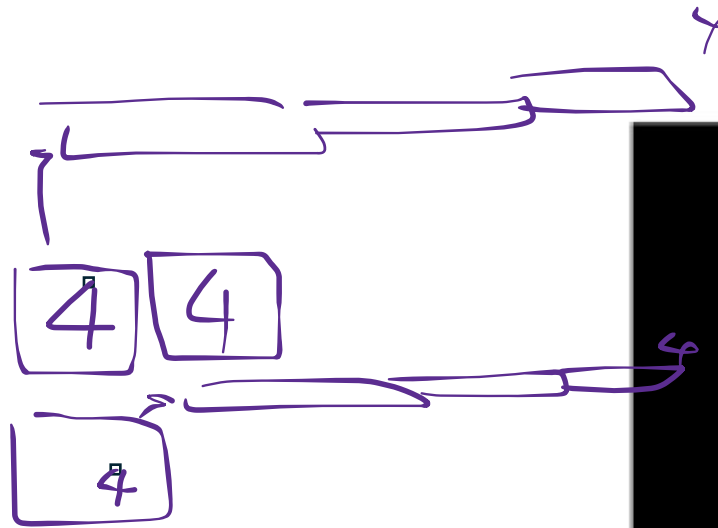
Dense layer



Convolution layer

Local Patterns

- edges
- textures
- simple shapes

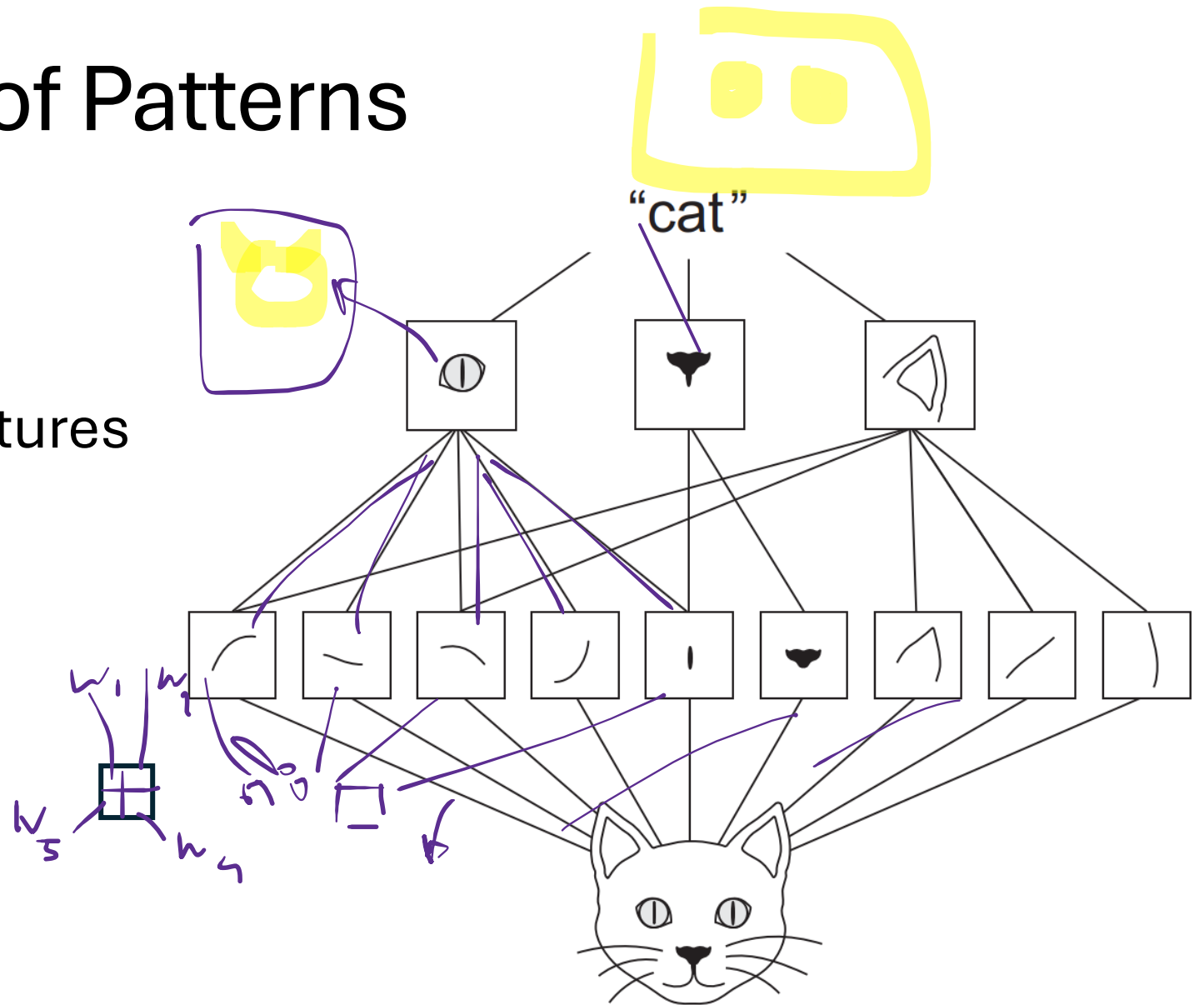
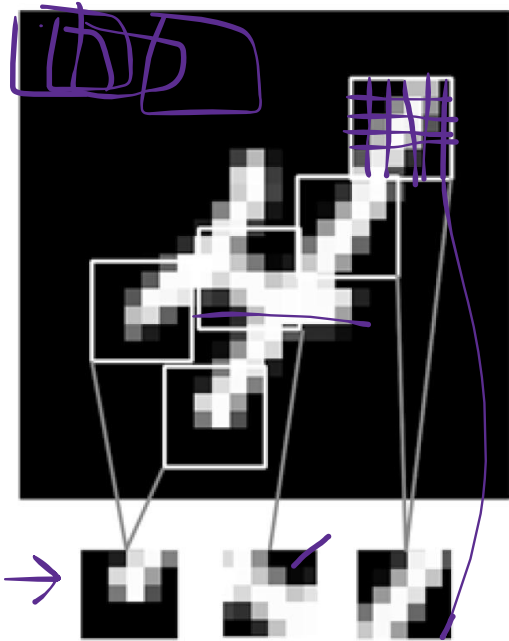


Translation Invariance

- translation invariant patterns
- pattern เดิม เจอได้ทุกตำแหน่ง

Spatial Hierarchy of Patterns

- simple → complex
- low-level → high-level features



Feature Maps และ Channels

- feature maps = 3D tensors
- (height, width, channels)

Filters และ Output Channels

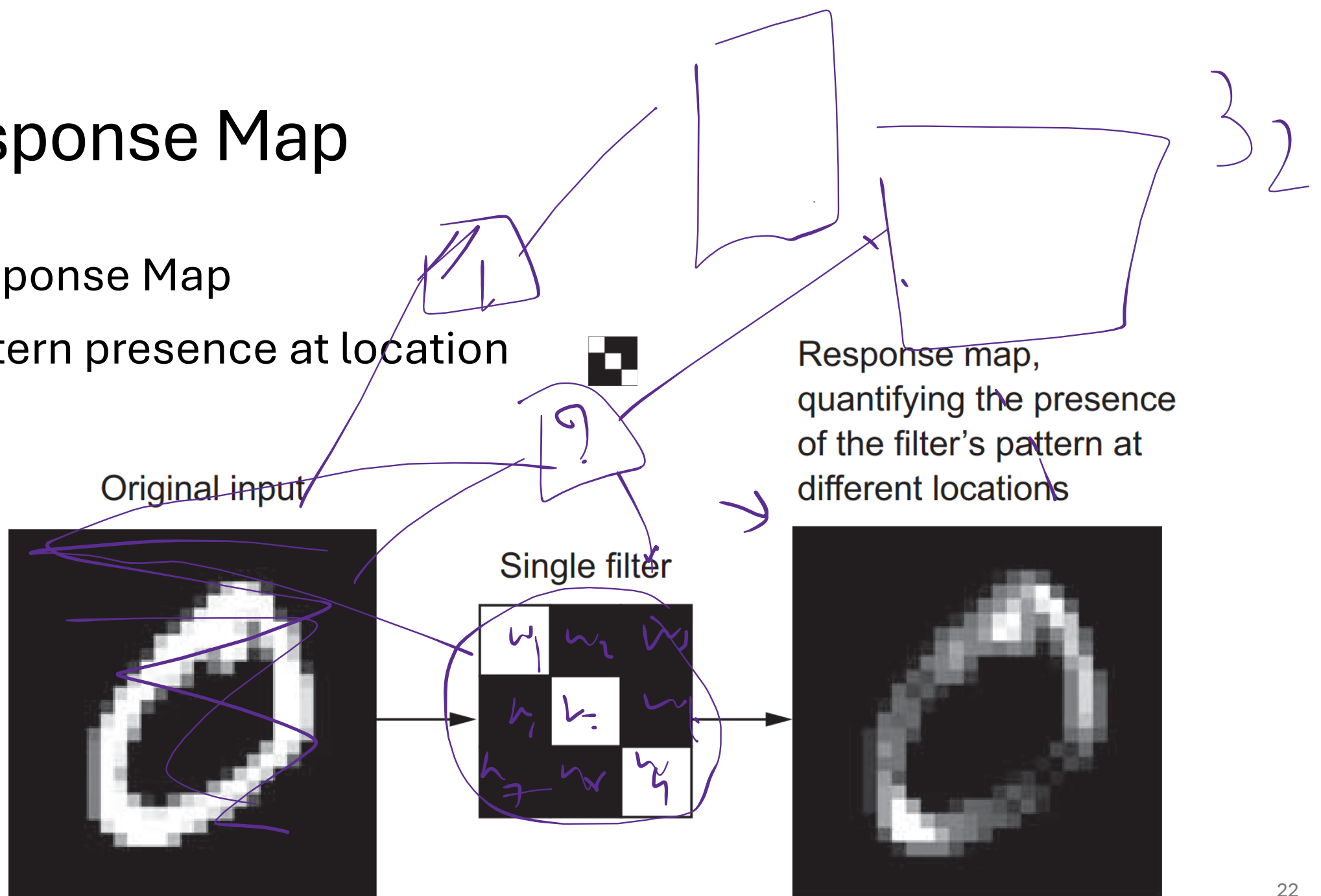
- 1 filter → 1 channel
- Conv2D(32) → 32 channels

model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)))

Layer (type)	Output Shape
conv2d_1 (Conv2D)	(None, 26, 26, 32)
maxpooling2d_1 (MaxPooling2D)	(None, 13, 13, 32)
conv2d_2 (Conv2D)	(None, 11, 11, 64)
maxpooling2d_2 (MaxPooling2D)	(None, 5, 5, 64)
conv2d_3 (Conv2D)	(None, 3, 3, 64)

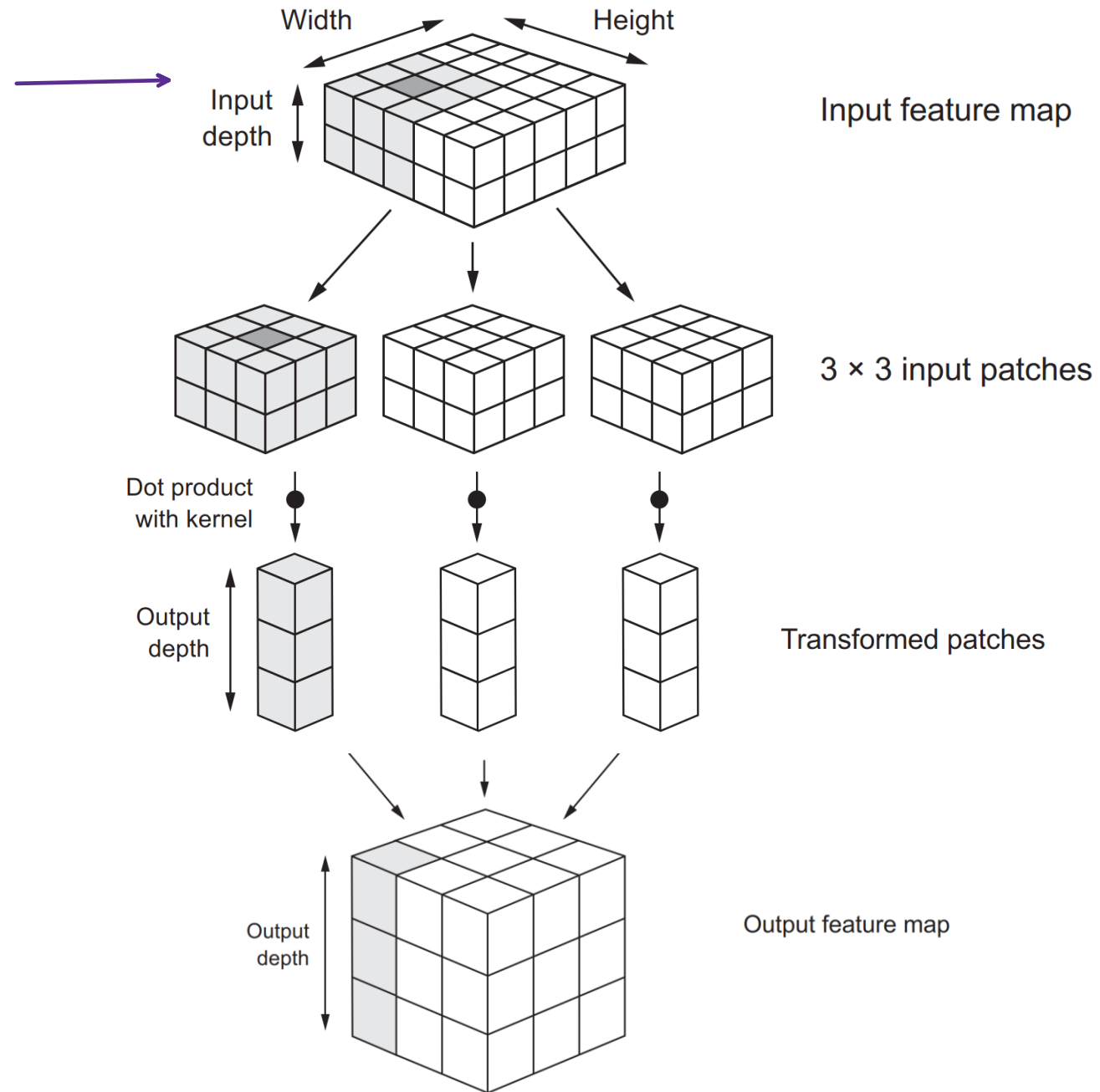
Response Map

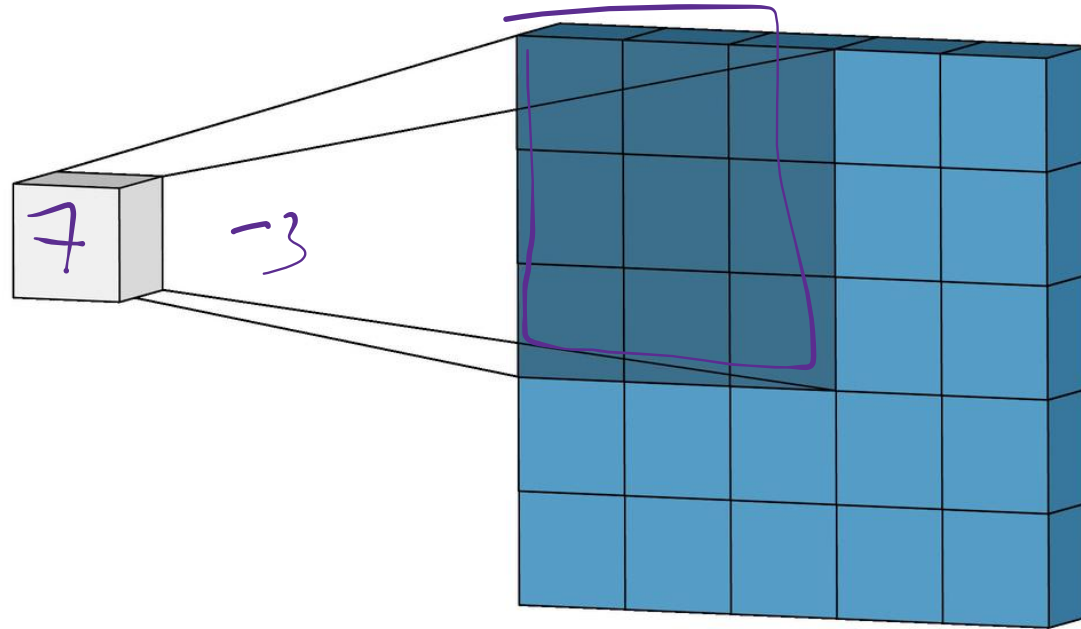
- Response Map
- Pattern presence at location



How Convolution Works

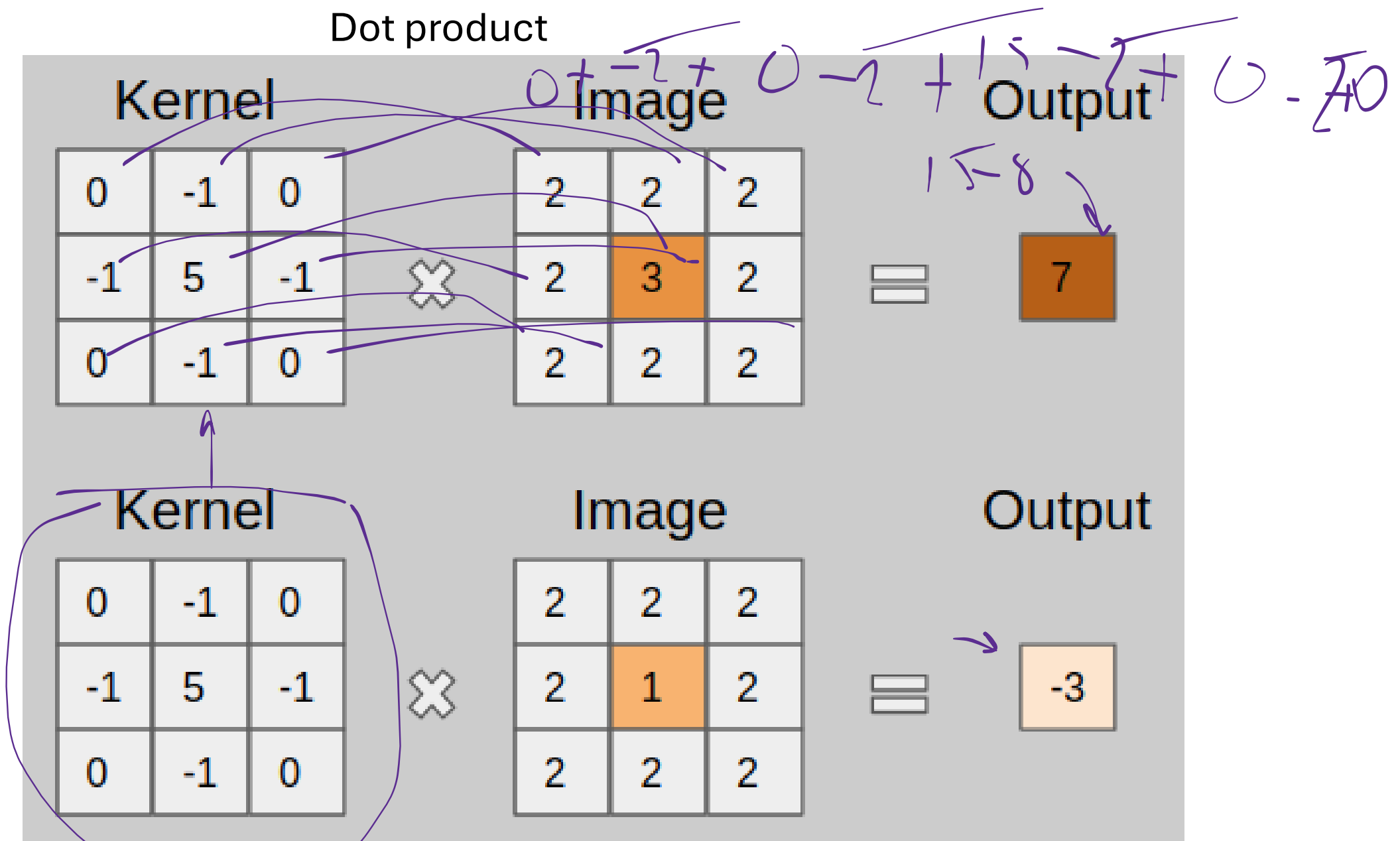
- sliding window
- same weights, different locations





<https://medium.com/@siddheshb008/understanding-convolution-neural-networks-a30211e12a06>

Dot product



Convolution



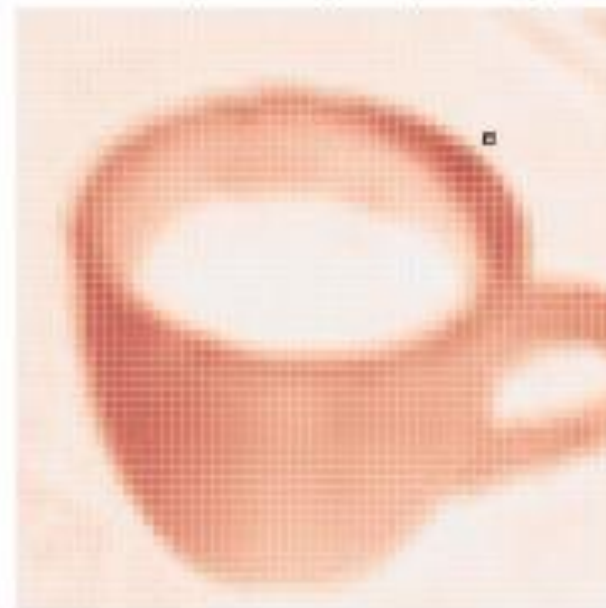
Input (64, 64)



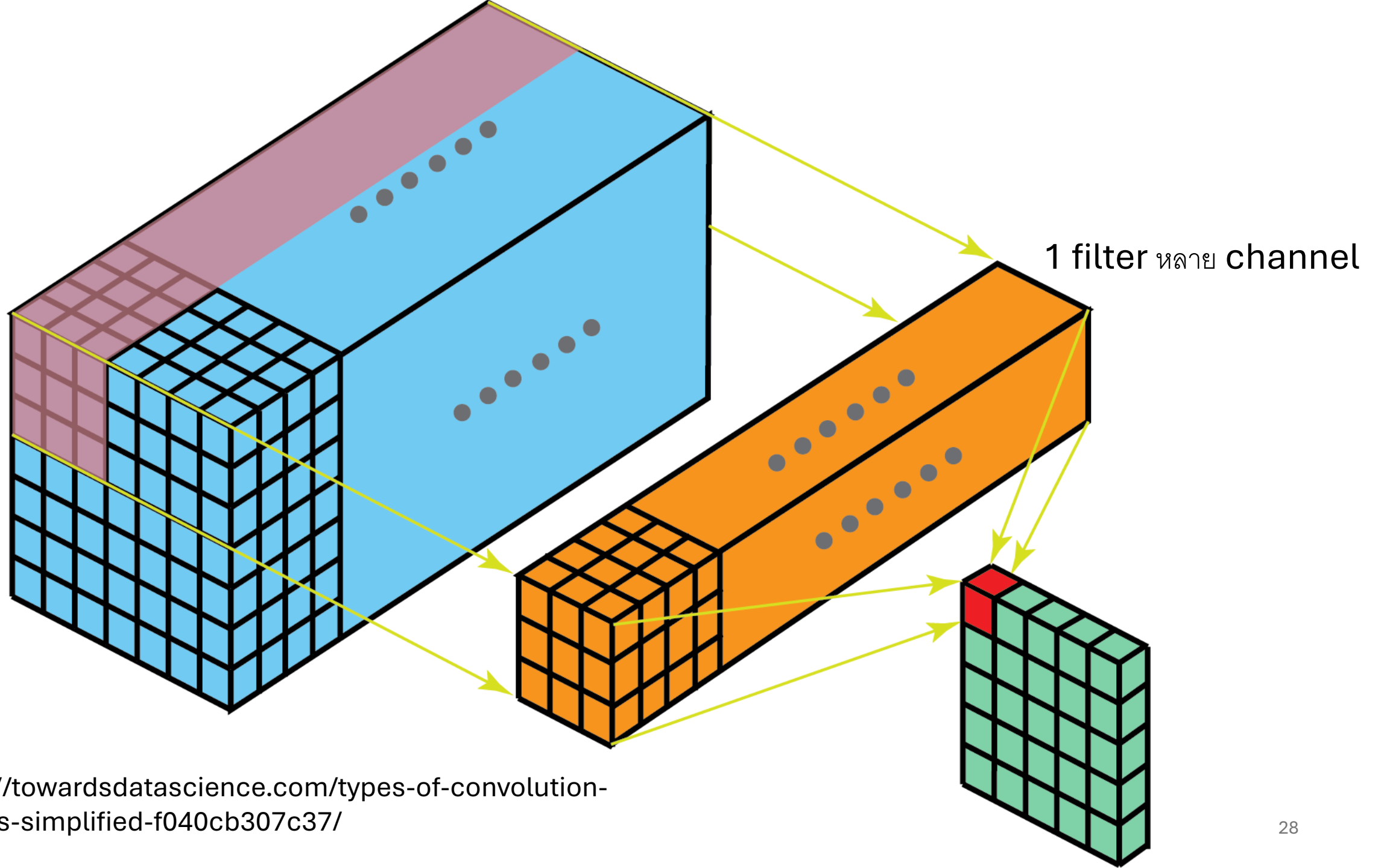
$$\begin{array}{r} \begin{array}{|c|} \hline 0.12 \\ \hline \end{array} + \begin{array}{|c|} \hline 0.12 \\ \hline \end{array} + \begin{array}{|c|} \hline 0.11 \\ \hline \end{array} + \\ \times \begin{array}{|c|} \hline -0.25 \\ \hline \end{array} \quad \times \begin{array}{|c|} \hline -0.22 \\ \hline \end{array} \quad \times \begin{array}{|c|} \hline 0.01 \\ \hline \end{array} \quad + \\ \begin{array}{|c|} \hline 0.13 \\ \hline \end{array} + \begin{array}{|c|} \hline 0.14 \\ \hline \end{array} + \begin{array}{|c|} \hline 0.13 \\ \hline \end{array} + \\ \times \begin{array}{|c|} \hline -0.17 \\ \hline \end{array} \quad \times \begin{array}{|c|} \hline -0.26 \\ \hline \end{array} \quad \times \begin{array}{|c|} \hline -0.03 \\ \hline \end{array} \quad + \\ \begin{array}{|c|} \hline 0.14 \\ \hline \end{array} + \begin{array}{|c|} \hline 0.14 \\ \hline \end{array} + \begin{array}{|c|} \hline 0.13 \\ \hline \end{array} = \end{array}$$

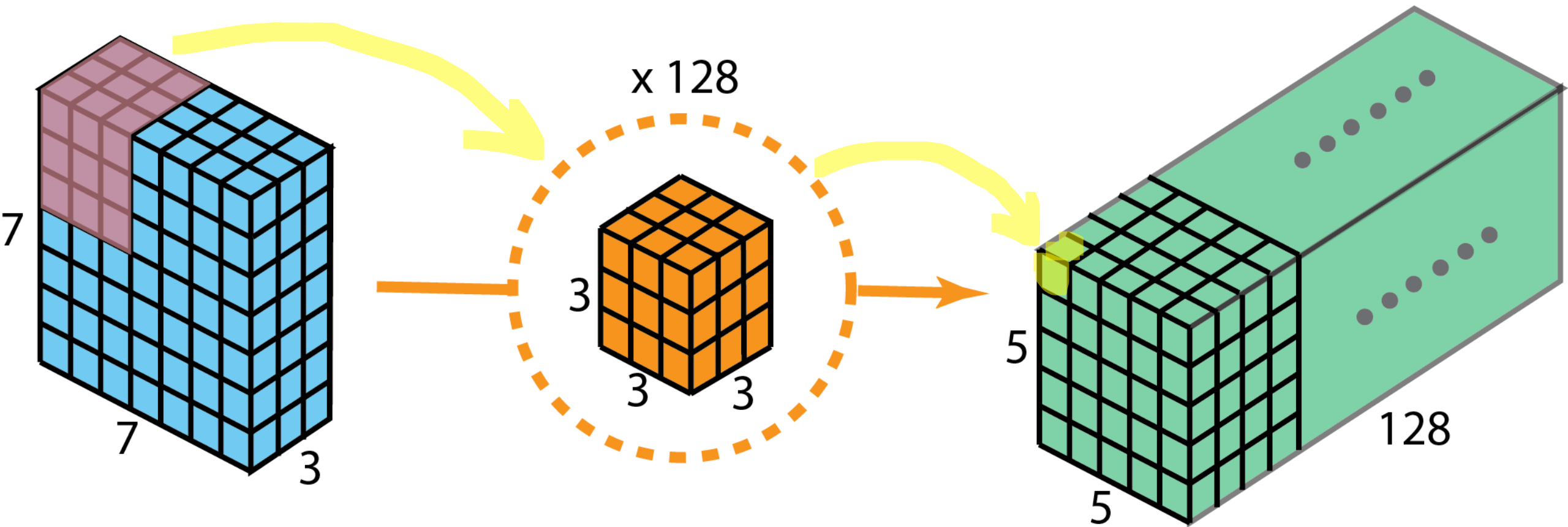
$$\begin{array}{|c|} \hline -0.18 \\ \hline \end{array}$$

Output (62, 62)

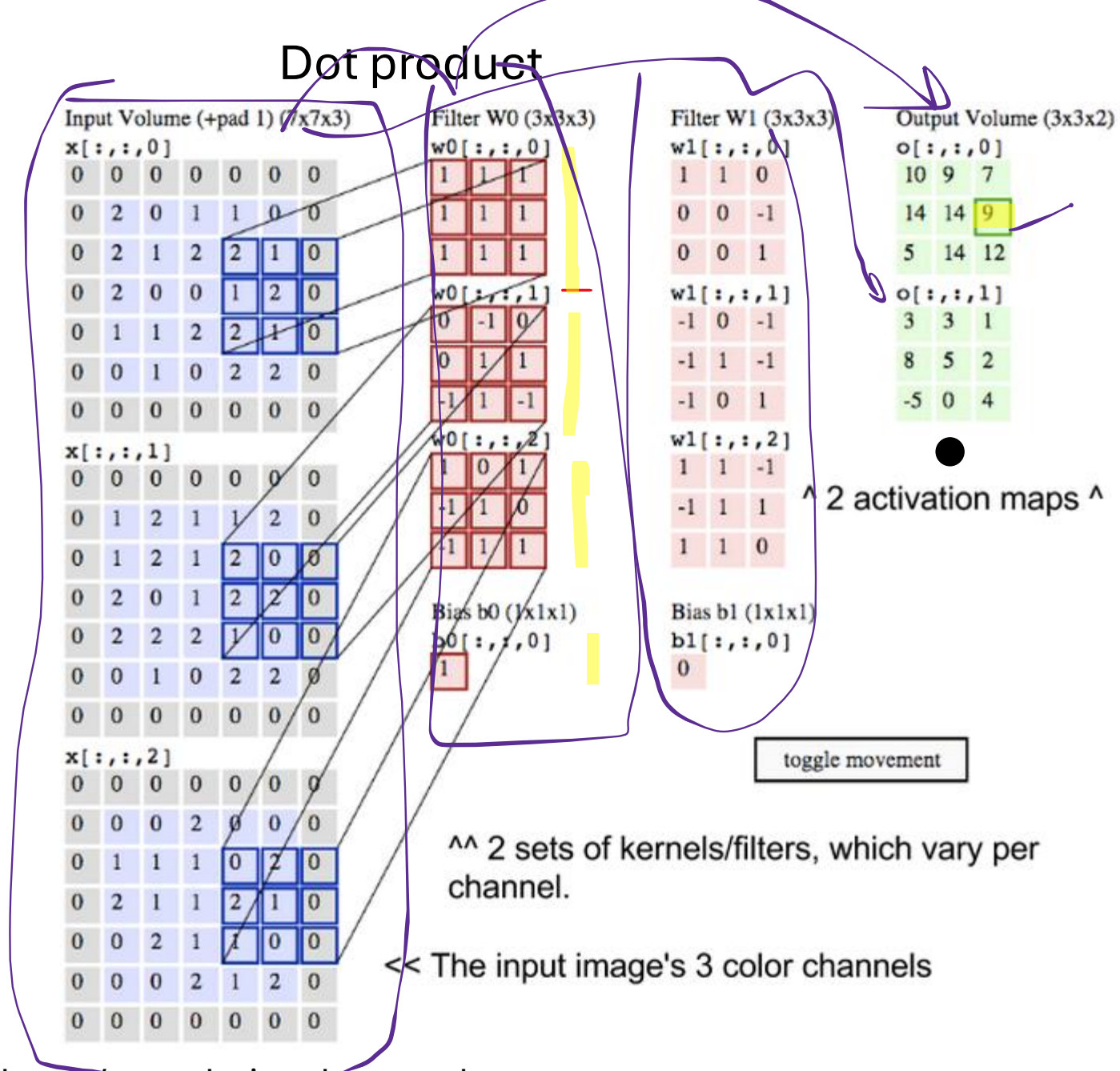


Hover over the matrices to change kernel position.

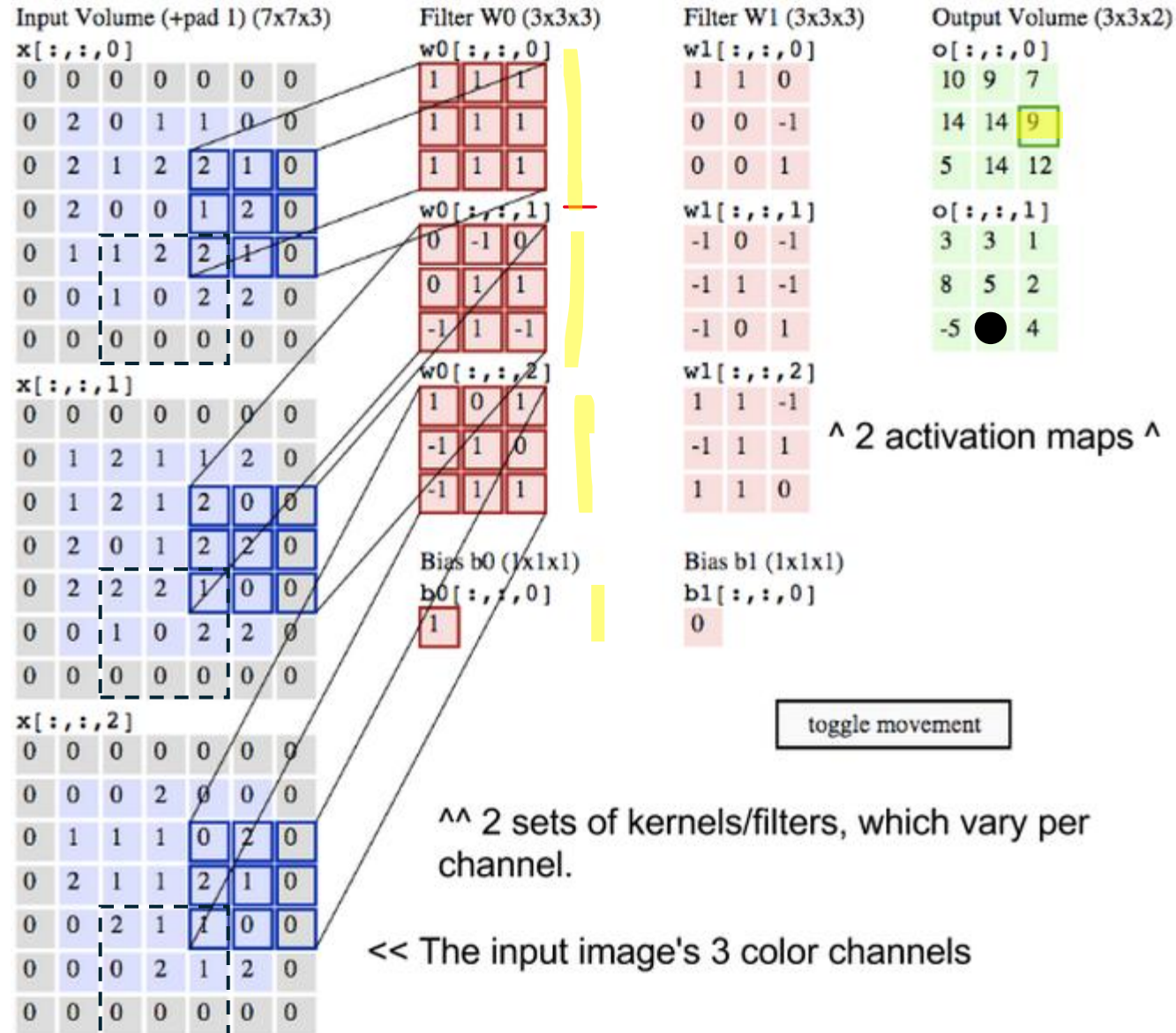




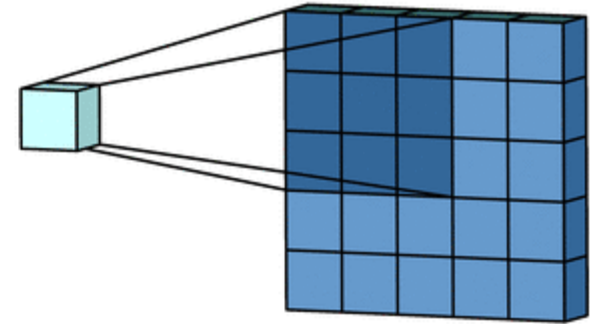
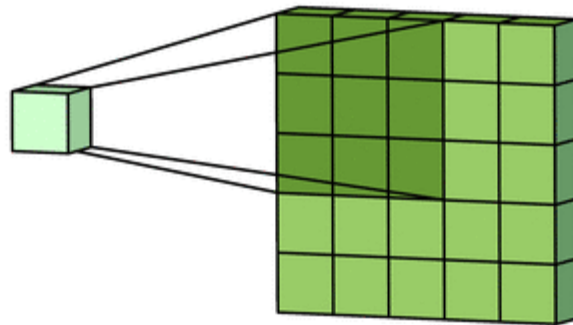
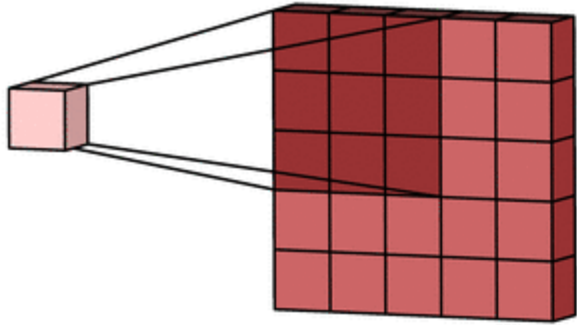
Dot product

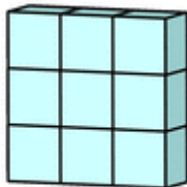
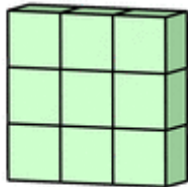
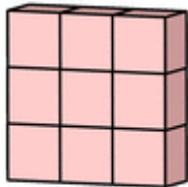


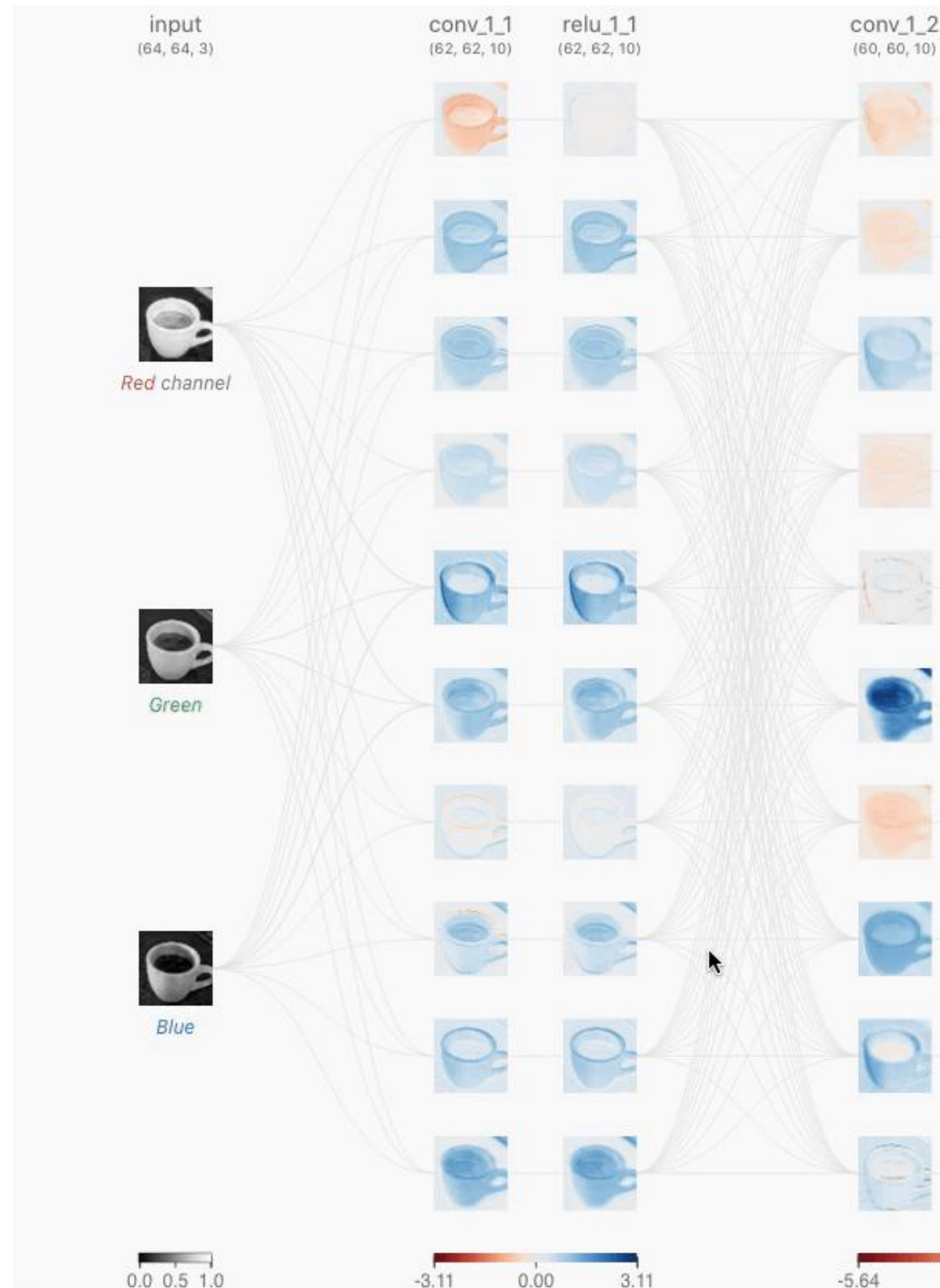
Dot product

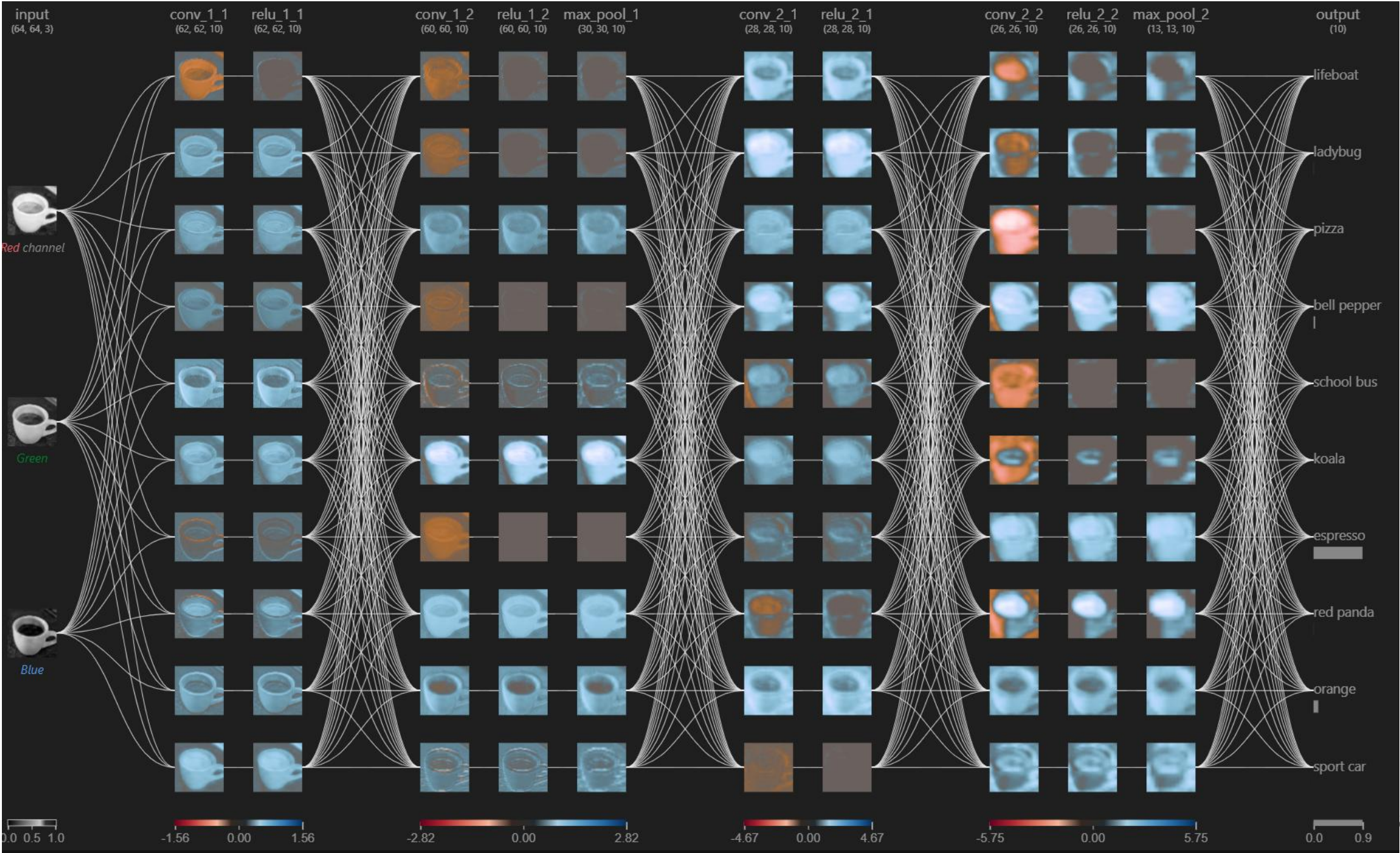


<https://medium.com/@siddheshb008/understanding-convolution-neural-networks-a30211e12a06>









Output Size

- border effects

```
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)))
```

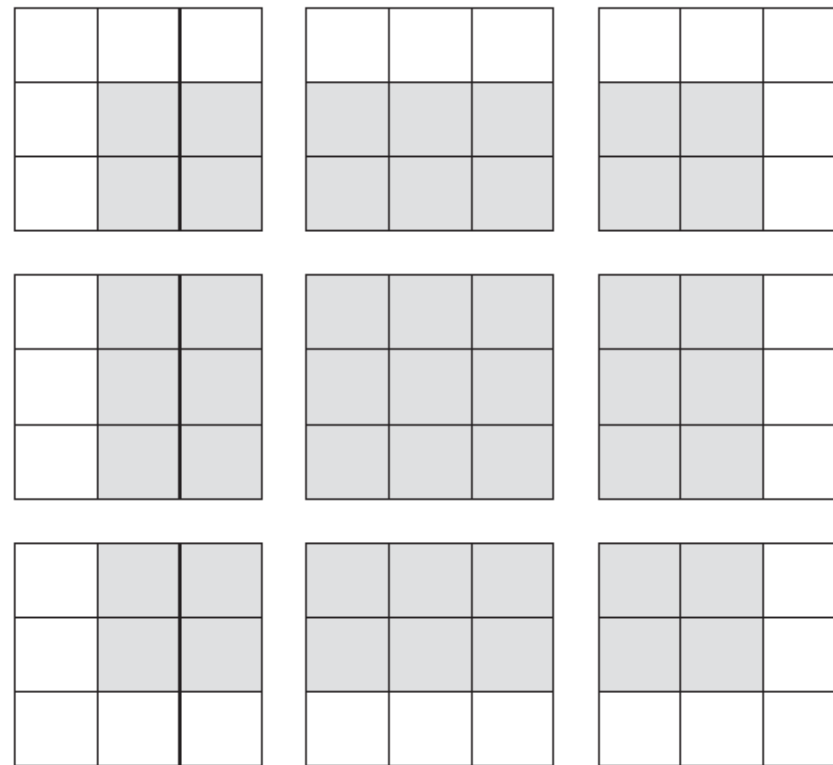
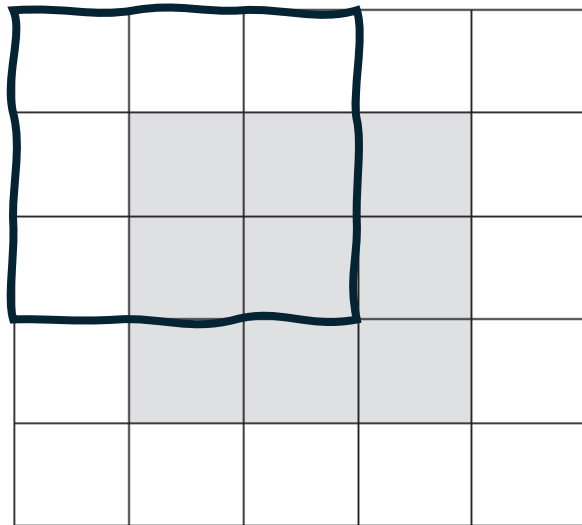
- padding

- strides

Layer (type)	Output Shape
conv2d_1 (Conv2D)	(None, 26, 26, 32)
maxpooling2d_1 (MaxPooling2D)	(None, 13, 13, 32)
conv2d_2 (Conv2D)	(None, 11, 11, 64)
maxpooling2d_2 (MaxPooling2D)	(None, 5, 5, 64)
conv2d_3 (Conv2D)	(None, 3, 3, 64)

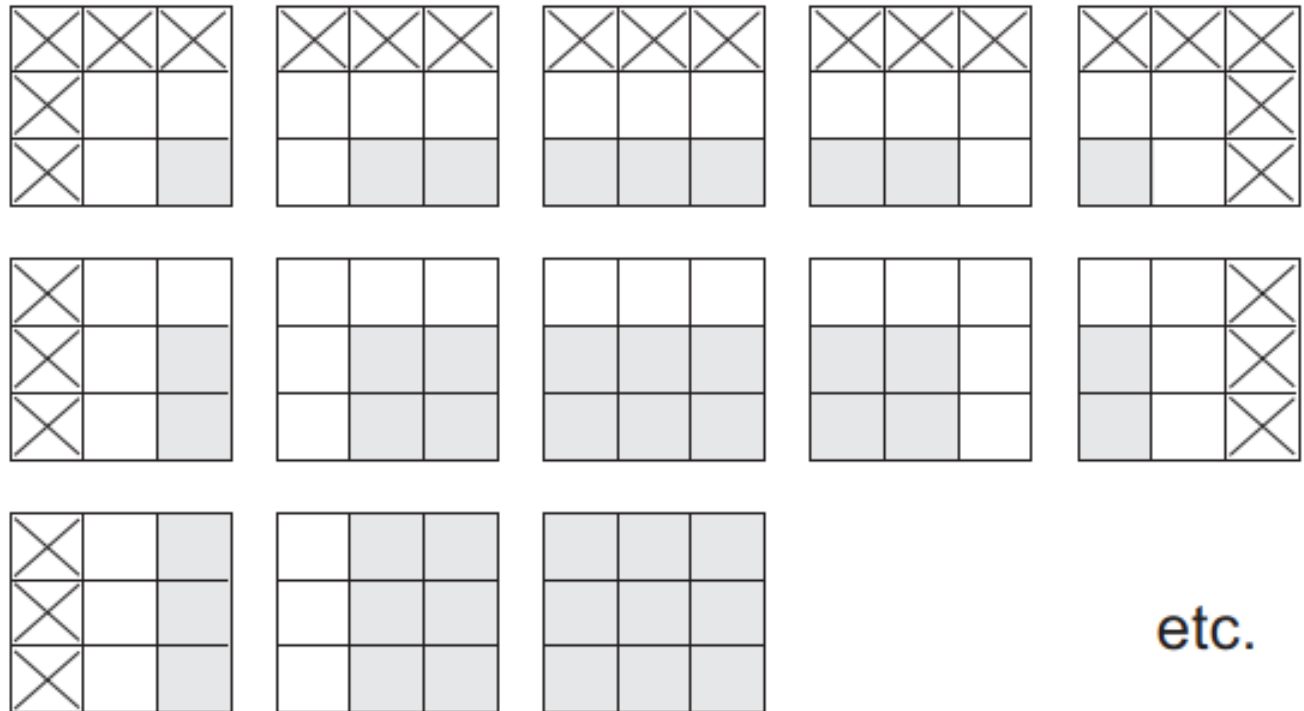
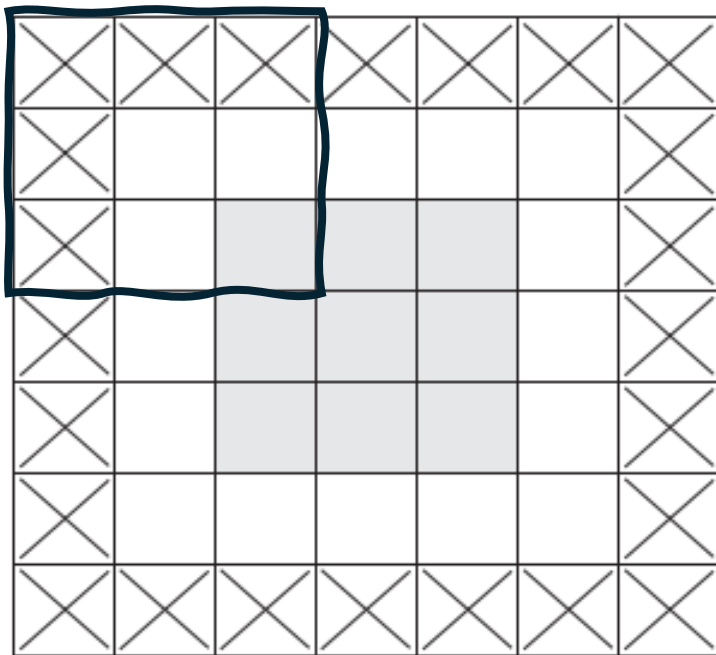
Border Effects

- valid positions only
- output เล็กกลง



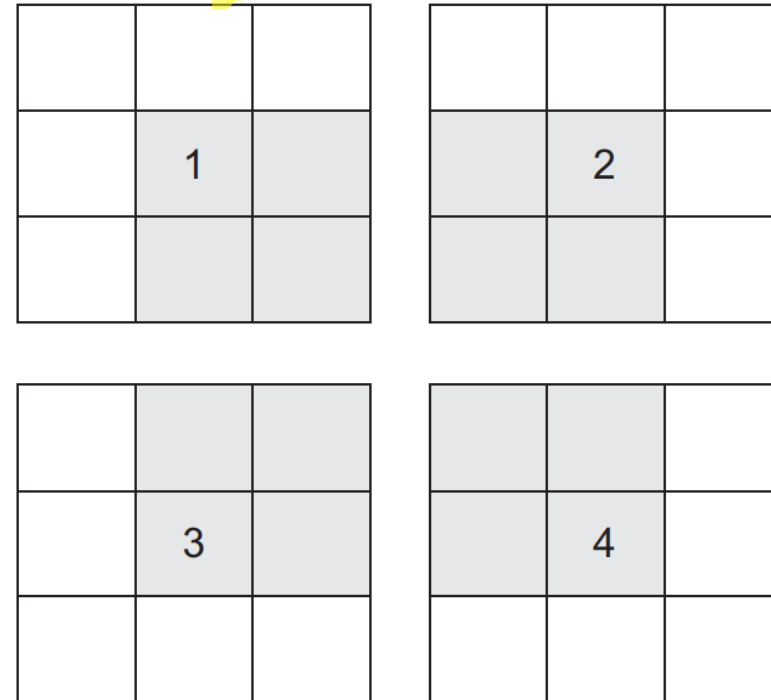
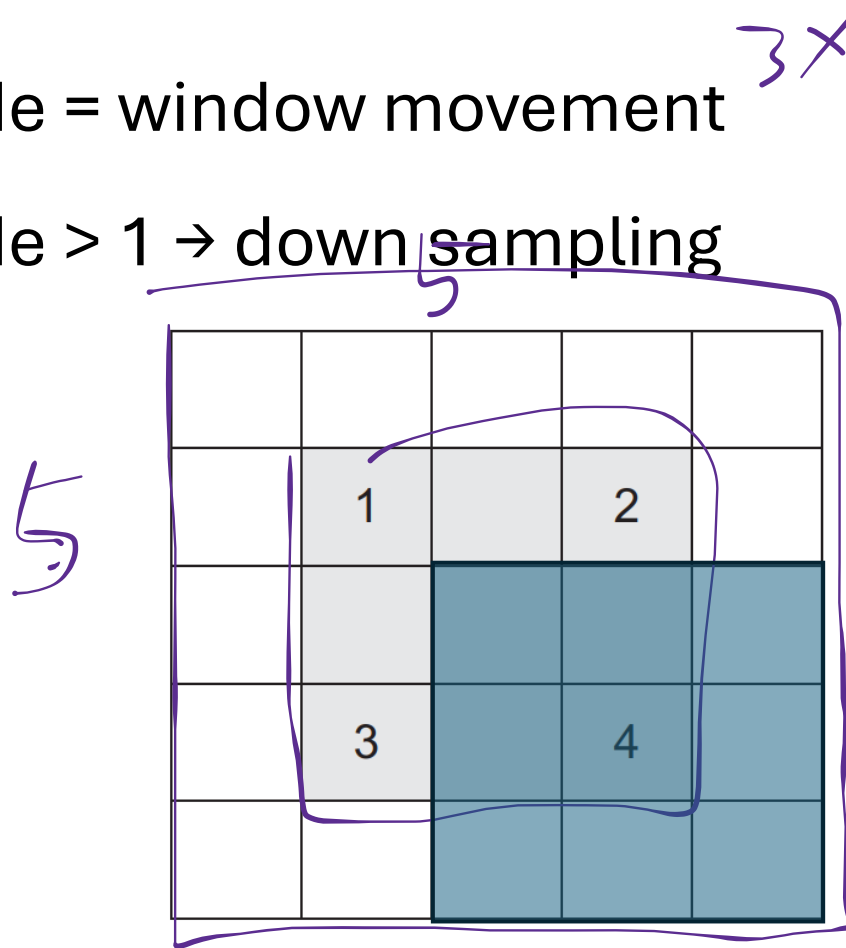
Padding

- padding = เพิ่มขอบ
- same vs valid (default)



Strides

- stride = window movement
- stride > 1 → down sampling



The Max-Pooling Operation

The max-pooling operation

Downsampling feature maps

สิ่งที่เกิดขึ้นกับ Feature Map

- ก่อน max pooling: 26×26
- หลัง max pooling: 13×13
- ขนาดลดลงครึ่งหนึ่ง

Layer (type)	Output Shape
conv2d_1 (Conv2D)	(None, 26, 26, 32)
maxpooling2d_1 (MaxPooling2D)	(None, 13, 13, 32)
conv2d_2 (Conv2D)	(None, 11, 11, 64)
maxpooling2d_2 (MaxPooling2D)	(None, 5, 5, 64)
conv2d_3 (Conv2D)	(None, 3, 3, 64)

การทำงาน Max Pooling

- sliding window
- เลือกค่า **max**
- ทำแยกตาม **channel**

Convolution vs Max pooling

- Convolution: learned transformation
- Max pooling: fixed operation

Window Size และ Stride ของ Max Pooling

- 2×2 window
- stride = 2
- downsample $\times 2$

Conv base model

```
model_no_max_pool = models.Sequential()  
model_no_max_pool.add(layers.Conv2D(32, (3, 3), activation='relu',  
input_shape=(28, 28, 1)))  
model_no_max_pool.add(layers.Conv2D(64, (3, 3),  
activation='relu'))  
model_no_max_pool.add(layers.Conv2D(64, (3, 3),  
activation='relu'))
```

Architecture Conv

```
model_no_max_pool.summary()
```

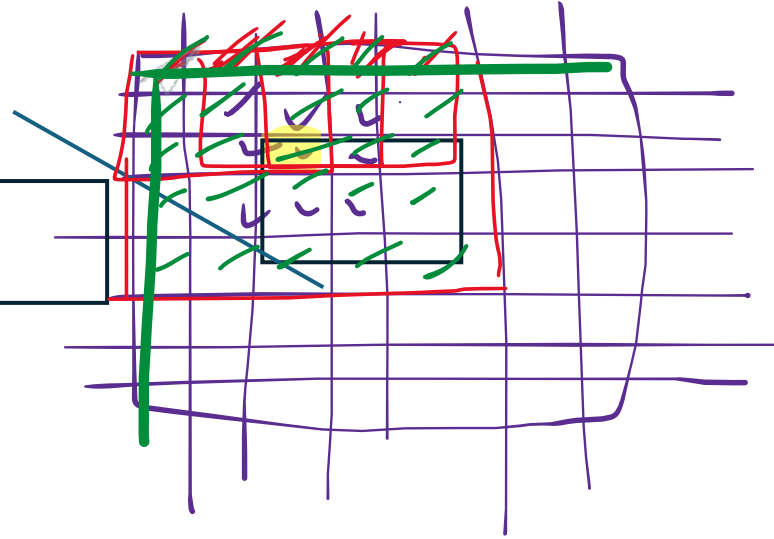
Layer (type)	Output Shape	Param #
conv2d_4 (Conv2D)	(None, 26, 26, 32)	320
conv2d_5 (Conv2D)	(None, 24, 24, 64)	18496
conv2d_6 (Conv2D)	(None, 22, 22, 64)	36928

=====
Total params: 55,744
Trainable params: 55,744
Non-trainable params: 0

ทำไมต้อง Downsample

ทำไมไม่เก็บ feature map ใหญ่ ๆ ไว้ตลอด

- ปัญหาที่ 1 – Spatial Hierarchy ไม่ชัด
 - receptive field เล็ก
 - high-level patterns ยังเล็ก



- ขั้นแรก CONV 3x3 map to 1 หน้าต่างมองภาพขนาด **3x3**
- ขั้นที่ 2 CONV 3x3 map to 1 เลื่อน kernel ขั้น input 2 ครั้ง หน้าต่างมองภาพขนาด **5x5**
- ขั้นที่ 3 CONV 3x3 map to 1 เลื่อน kernel ขั้น input 2 ครั้ง หน้าต่างมองภาพขนาด **7x7**

หน้าต่างโตช้ามาก

ทำไมต้อง Downsample

ทำไมไม่เก็บ feature map ใหญ่ ๆ ไว้ตลอด

- ปัญหาที่ 2 – จำนวนพารามิเตอร์มหาศาล

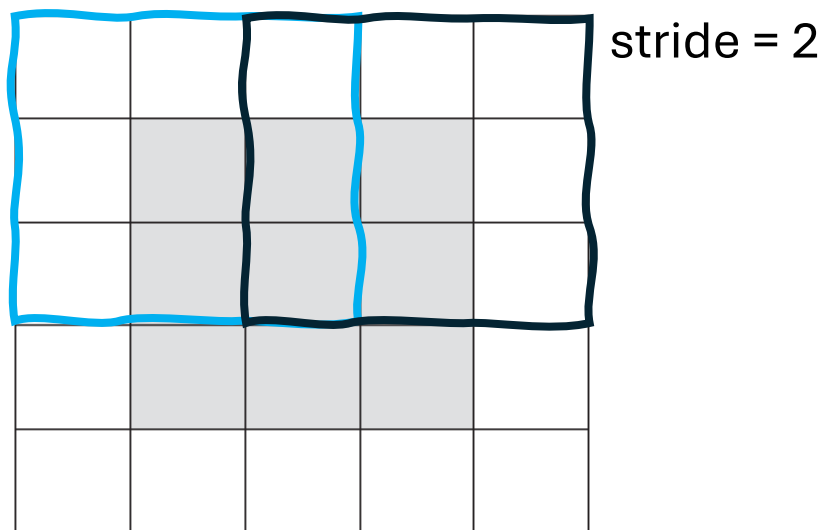
- feature map ใหญ่
- flatten แล้ว Dense ใหญ่มาก
- Overfitting

*** feature map สุดท้ายมีขนาด 22x22x64

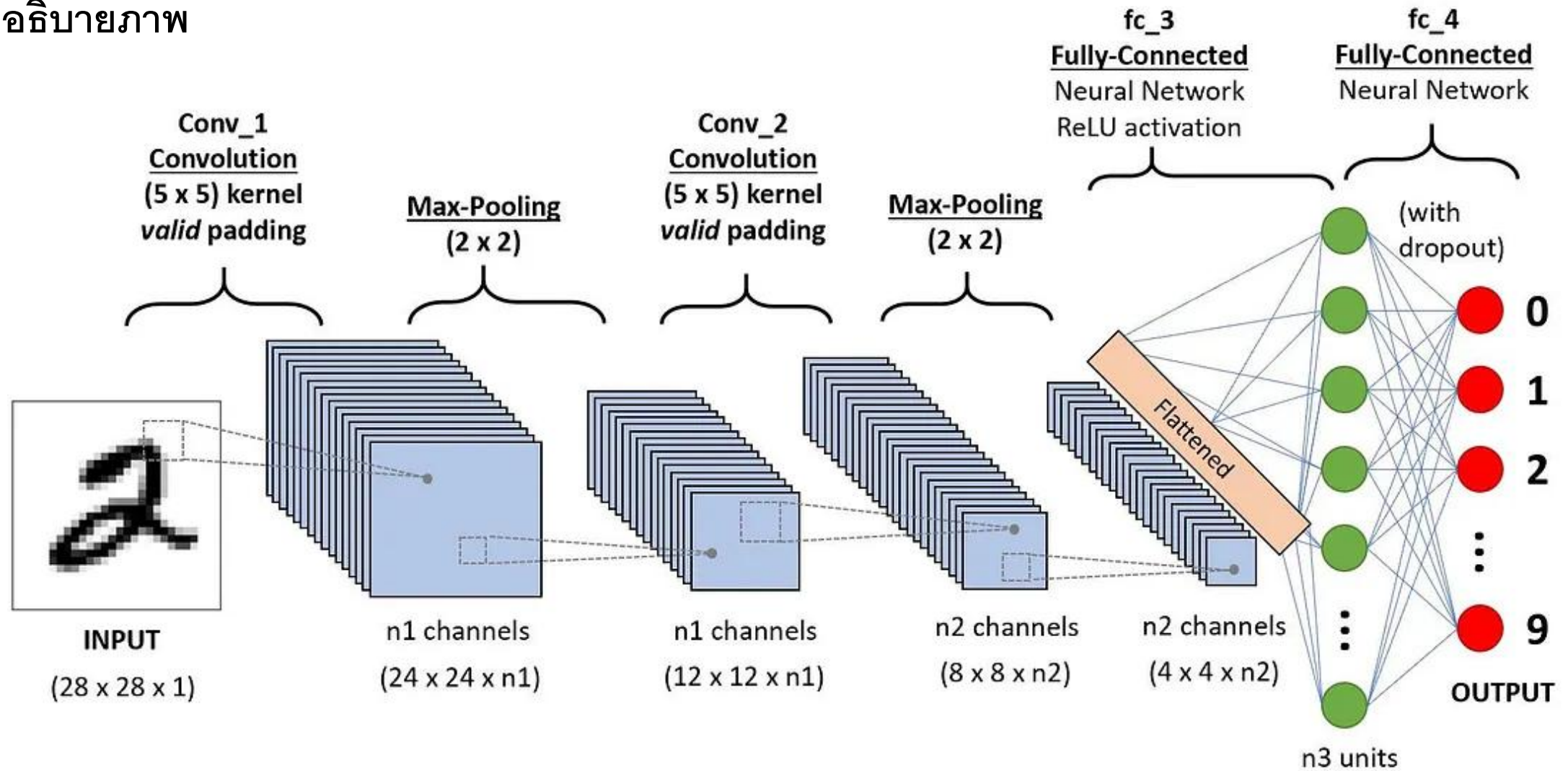
Layer (type)	Output Shape
=====	=====
conv2d_4 (Conv2D)	(None, 26, 26, 32)
conv2d_5 (Conv2D)	(None, 24, 24, 64)
conv2d_6 (Conv2D)	(None, 22, 22, 64)
=====	=====

ทางเลือกอื่นแทน Max Pooling

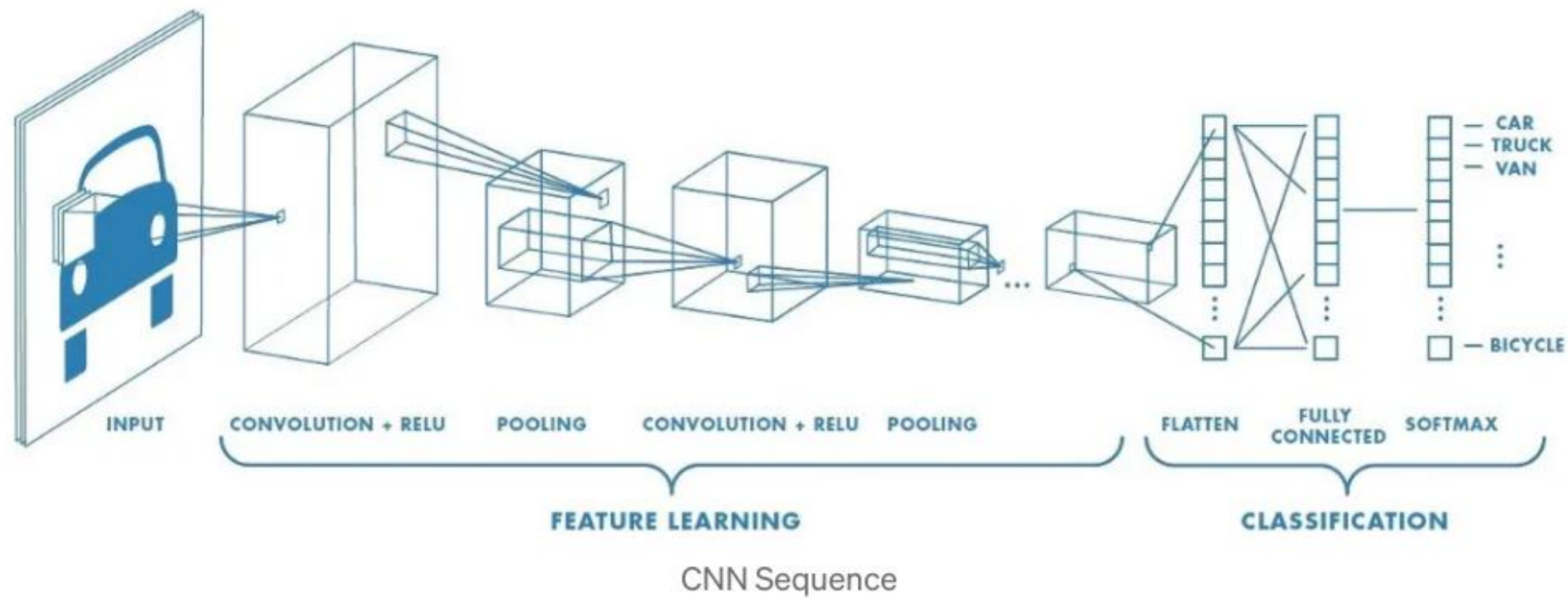
- strided convolution
- average pooling



อธิบายภาพ



อธิบายภาพ



Training a convnet from scratch on a small dataset

Training a ConvNet from Scratch on a Small Dataset

- Small dataset is a common real-world problem
- “Few samples” $\neq$ extremely small
- Example task: Dogs vs Cats image classification

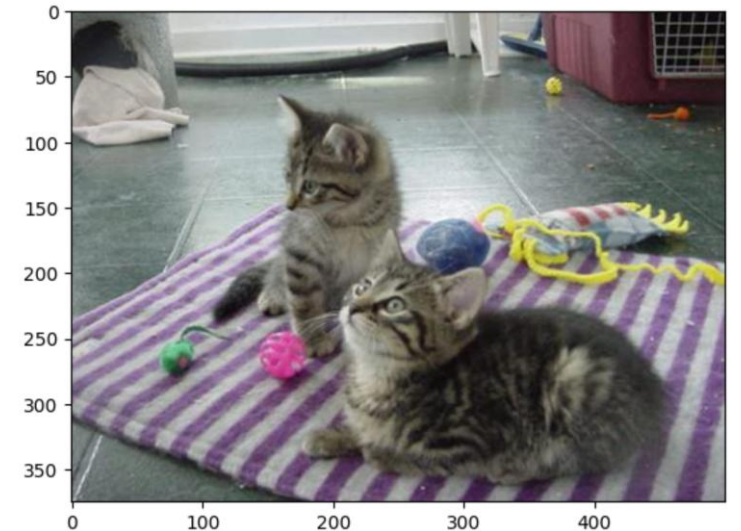
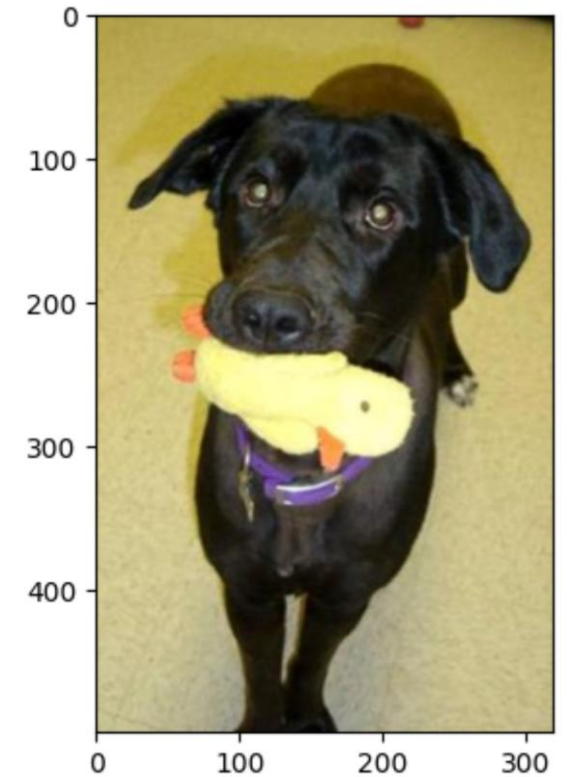
Dataset Setup in This Example

Total images: 4,000

- 2,000 cats
- 2,000 dogs

Data split:

- 2,000 training
- 1,000 validation
- 1,000 test



Strategy

- Train a small ConvNet from scratch
- No regularization → set a baseline
- Identify the main problem: overfitting

Expected Performance Progression

- From scratch, no regularization → ~71% accuracy
- With data augmentation → ~82%

Later sections:

- Feature extraction → 90–96%
- Fine-tuning → ~97%

Deep Learning and Small-Data Problems

- Common belief: deep learning needs lots of data
- This is partly true
- Especially for high-dimensional inputs (e.g. images)

What “Lots of Data” Really Means

Relative to:

- Model size
- Model depth
- Task complexity

Few hundred samples can be enough

- If the model is small
- If the task is simple
- If regularization is used

Why ConvNets Are Data-Efficient

- Learn local features
- Translation-invariant
- Well-suited for perceptual problems

Repurposability of Deep Learning Models

- Models are highly reusable
- Knowledge learned from large datasets
- Can be transferred to new tasks

Downloading the Data

- Dataset: Dogs vs Cats (Kaggle)
- Not packaged with Keras
- Originally released in 2013

<https://www.kaggle.com/datasets/karakaggle/kaggle-cat-vs-dog-dataset>

```
import kagglehub

# Download latest version
path = kagglehub.dataset_download("karakaggle/kaggle-cat-vs-dog-dataset")

print("Path to dataset files:", path)
```

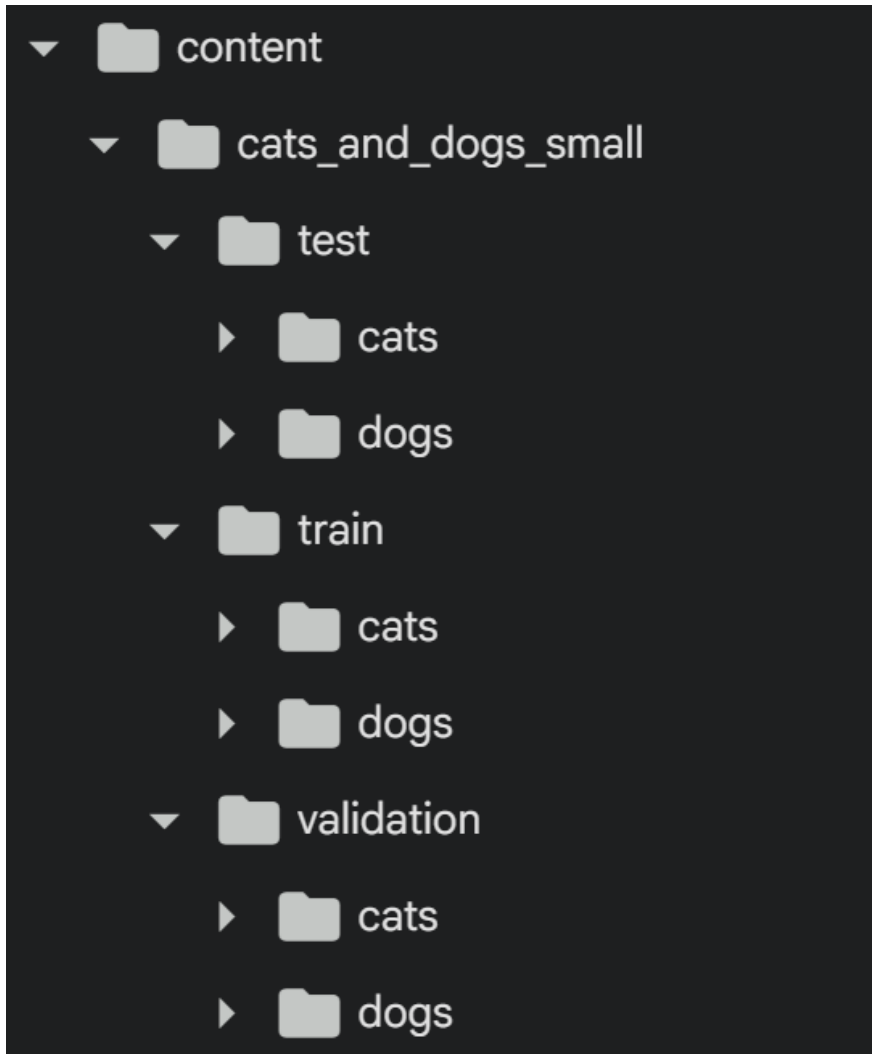
Raw Dataset Characteristics

- 25,000 total images
- JPEG color images
- Heterogeneous sizes and appearances

Creating a Smaller Dataset

- Create a new dataset with:
- Training set
- Validation set
- Test set
- Balanced binary classification

Directory Structure



```
# Path to the directory where the original dataset was uncompressed
```

```
import os, shutil
```

```
original_dataset_dir = os.path.join(path, "kagglecatsanddogs_3367a", "PetImages")
```

```
#Cats data set
```

```
cat_src_dir = os.path.join(original_dataset_dir, 'Cat')
```

```
dog_src_dir = os.path.join(original_dataset_dir, 'Dog')
```

```
print(os.path.exists(cat_src_dir)) # check true or not
```

```
print(os.path.exists(dog_src_dir)) # check true or not
```

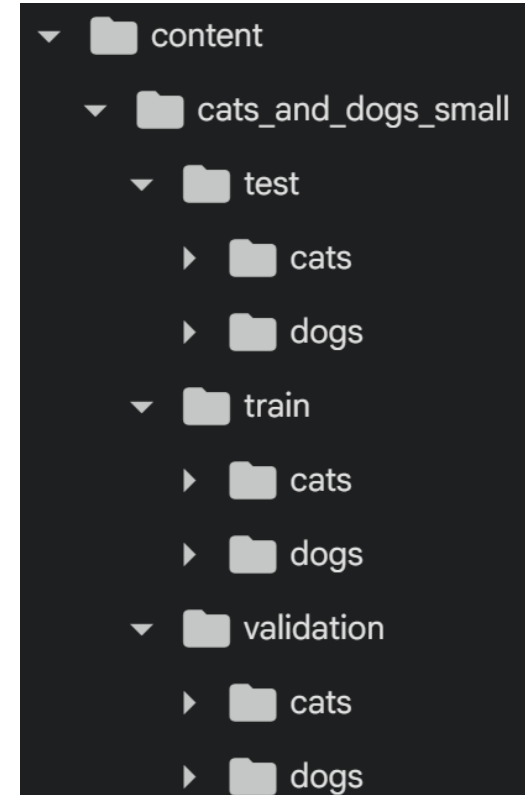
```
# Directory where you'll store your smaller dataset
base_dir = "/content/cats_and_dogs_small"
if os.path.exists(base_dir): # clear folder
    shutil.rmtree(base_dir)

os.makedirs(base_dir, exist_ok=True)

train_dir = os.path.join(base_dir, 'train')
os.makedirs(train_dir, exist_ok=True)

test_dir = os.path.join(base_dir, 'test')
os.makedirs(test_dir, exist_ok=True)

validation_dir = os.path.join(base_dir, 'validation')
os.makedirs(validation_dir, exist_ok=True)
```



```
train_cats_dir = os.path.join(train_dir, 'cats')  
os.makedirs(train_cats_dir, exist_ok=True)
```

```
train_dogs_dir = os.path.join(train_dir, 'dogs')  
os.makedirs(train_dogs_dir, exist_ok=True)
```

```
validation_cats_dir = os.path.join(validation_dir, 'cats')  
os.makedirs(validation_cats_dir, exist_ok=True)
```

```
validation_dogs_dir = os.path.join(validation_dir, 'dogs')  
os.makedirs(validation_dogs_dir, exist_ok=True)
```

```
test_cats_dir = os.path.join(test_dir, 'cats')  
os.makedirs(test_cats_dir, exist_ok=True)
```

```
test_dogs_dir = os.path.join(test_dir, 'dogs')  
os.makedirs(test_dogs_dir, exist_ok=True)
```

แสดงรูป

```
import matplotlib.pyplot as plt
from PIL import Image
img_path = os.path.join(dog_src_dir, dog_files[500])
img = Image.open(img_path)
plt.imshow(img)
plt.show()
print("Image size (width, height):", img.size)
import numpy as np
print("Image shape (height, width, channels)
if converted to numpy array:", np.array(img).shape)
```

```
cat_files = [f for f in os.listdir(cat_src_dir)]
dog_files = [f for f in os.listdir(dog_src_dir)]
```

```
train_cats = cat_files[:1000]
train_dogs = dog_files[:1000]
```

```
val_cats    = cat_files[1000:1500]
val_dogs    = dog_files[1000:1500]
```

```
test_cats   = cat_files[1500:2000]
test_dogs   = dog_files[1500:2000]
```

แบบใหม่ def

```
fnames = ['cat.{}.jpg'.format(i) for i in range(1000)]
for fname in fnames:
    src = os.path.join(original_dataset_dir, fname)
    dst = os.path.join(train_cats_dir, fname)
    shutil.copyfile(src, dst)
```

```
def copy_files(file_list, src_dir, dst_dir):
    for fname in file_list:
        src = os.path.join(src_dir, fname)
        dst = os.path.join(dst_dir, fname)
        shutil.copyfile(src, dst)
```

```
copy_files(train_cats, cat_src_dir, os.path.join(base_dir, "train", "cats"))
copy_files(val_cats, cat_src_dir, os.path.join(base_dir, "validation", "cats"))
copy_files(test_cats, cat_src_dir, os.path.join(base_dir, "test", "cats"))

copy_files(train_dogs, dog_src_dir, os.path.join(base_dir, "train", "dogs"))
copy_files(val_dogs, dog_src_dir, os.path.join(base_dir, "validation", "dogs"))
copy_files(test_dogs, dog_src_dir, os.path.join(base_dir, "test", "dogs"))

print("Dataset ready at:", base_dir)
print(len(train_cats), len(val_cats), len(test_cats))
print(len(train_dogs), len(val_dogs), len(test_dogs))
```

เช็คว่าจำนวนข้อมูลเท่ากัน

```
for name in ["train", "validation", "test"]:
    print(os.path.join(base_dir, name, "cats"))
    print(len(os.listdir(os.path.join(base_dir, name, "cats"))))

    print(os.path.join(base_dir, name, "dogs"))
    print(len(os.listdir(os.path.join(base_dir, name, "dogs"))))
```

- Training: 2,000 images
- Validation: 1,000 images
- Test: 1,000 images

Building network

ในตัวอย่างก่อนหน้านี้ เราได้สร้าง **small convnet** สำหรับ **MNIST**

- โครงสร้างพื้นฐานยังคงเหมือนเดิม คือ
- Conv2D (activation = ReLU)
- MaxPooling2D

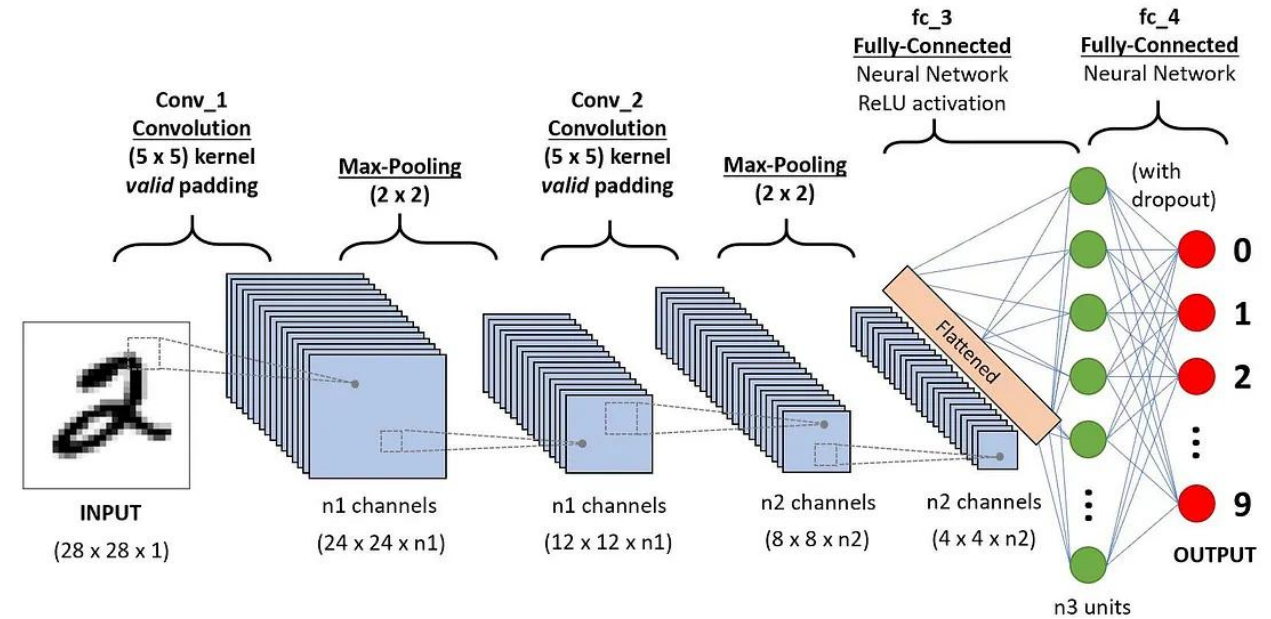
ปัญหา **dogs vs cats**:

- input image ใหญ่กว่า
- ลักษณะข้อมูลซับซ้อนกว่า
- ดังนั้นต้อง ขยายขนาด **network**

ConvNet Structure Pattern

General ConvNet structure

- ConvNet ประกอบด้วยการสลับ:
 - Conv2D (feature extraction)
 - MaxPooling2D (downsampling)
- เพิ่มจำนวน Conv + Pooling stages
- เป้าหมาย:
 - เพิ่ม **model capacity**
 - ลดขนาด feature maps ก่อน Flatten



Feature Map Size Evolution

Spatial size vs depth

- Input: $150 \times 150 \times 3$
- ก่อน Flatten: $7 \times 7 \times 128$

แนวโน้ม:

- spatial size (ขนาดภาพ) ลดลง
- depth (channels) เพิ่มขึ้น

Output Layer for Binary Classification

ปัญหา: binary classification

- Output layer:
 - Dense(1)
 - activation = sigmoid
- Output value $\in (0, 1)$

Model Architecture

```
model = models.Sequential()  
model.add(layers.Conv2D(32, (3,3), activation='relu', input_shape=(150,150,3)))  
model.add(layers.MaxPooling2D((2,2)))  
model.add(layers.Conv2D(64, (3,3), activation='relu'))  
model.add(layers.MaxPooling2D((2,2)))  
model.add(layers.Conv2D(128, (3,3), activation='relu'))  
model.add(layers.MaxPooling2D((2,2)))  
model.add(layers.Conv2D(128, (3,3), activation='relu'))  
model.add(layers.MaxPooling2D((2,2)))  
model.add(layers.Flatten())  
model.add(layers.Dense(512, activation='relu'))  
model.add(layers.Dense(1, activation='sigmoid'))
```

```
>>> model.summary()
```

Layer (type)	Output Shape	Param #
=====		
conv2d_1 (Conv2D)	(None, 148, 148, 32)	896
maxpooling2d_1 (MaxPooling2D)	(None, 74, 74, 32)	0
conv2d_2 (Conv2D)	(None, 72, 72, 64)	18496
maxpooling2d_2 (MaxPooling2D)	(None, 36, 36, 64)	0
conv2d_3 (Conv2D)	(None, 34, 34, 128)	73856
maxpooling2d_3 (MaxPooling2D)	(None, 17, 17, 128)	0
conv2d_4 (Conv2D)	(None, 15, 15, 128)	147584
maxpooling2d_4 (MaxPooling2D)	(None, 7, 7, 128)	0
flatten_1 (Flatten)	(None, 6272)	0
dense_1 (Dense)	(None, 512)	3211776
dense_2 (Dense)	(None, 1)	513

=====

Total params: 3,453,121

Trainable params: 3,453,121

Non-trainable params: 0

Parameter distribution

- Conv layers:
- parameters เพิ่มขึ้นตาม kernel และ channels
- Dense layers:
- parameters จำนวนมากที่สุด

Model Compilation

```
model.compile(  
    loss='binary_crossentropy',  
    optimizer=optimizers.RMSprop(lr=1e-4),  
    metrics=['accuracy']  
)
```

Data preprocessing

1. Read image files
2. Decode JPEG → RGB
3. Convert to floating-point tensors
4. Rescale pixel values to [0,1]

```
train_generator = train_datagen.flow_from_directory(  
    train_dir,  
    target_size=(150,150),  
    batch_size=20,  
    class_mode='binary'  
)
```

ImageDataGenerator

- Utility สำหรับ image preprocessing
- ทำงานอัตโนมัติ:
- rescale
- resize
- batching

`ImageDataGenerator(rescale=1./255)`

```
# from keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.preprocessing.image import ImageDataGenerator
train_datagen = ImageDataGenerator(rescale=1./255) # ตั้งค่าไว้ก่อน ยังไม่หารจริง
test_datagen = ImageDataGenerator(rescale=1./255)
```

```
train_generator = train_datagen.flow_from_directory(
    train_dir,
    target_size=(150, 150),
    batch_size=20,
    class_mode='binary')
```

```
validation_generator = test_datagen.flow_from_directory(
    validation_dir,
    target_size=(150, 150),
    batch_size=20,
    class_mode='binary')
```

Python Generators

- Generator = iterator
- ใช้ `yield` (จำค่าเดิมได้ ไม่ใช่ `return`)
- สร้าง `batch` แบบไม่สิ้นสุด

```
def generator():  
    i = 0  
    while True:  
        i += 1  
        yield i
```

```
for item in generator():  
    print(item)  
    if item > 4:  
        Break
```

It prints this:

```
1  
2  
3  
4  
5
```

Generator Output

```
train_generator =  
train_datagen.flow_from_directory(  
    train_dir,  
    target_size=(150, 150),  
    batch_size=20,  
    class_mode='binary')
```

```
for data_batch, labels_batch in train_generator:  
    print('data batch shape:', data_batch.shape)  
    print('labels batch shape:', labels_batch.shape)  
    break
```

```
>>> data batch shape: (20, 150, 150, 3)  
>>> labels batch shape: (20,)
```

Training with fit

```
history = model.fit(  
    train_generator,  
    steps_per_epoch=100,  
    epochs=30,  
    validation_data=validation_generator,  
    validation_steps=50)
```

$$20 \times 100 = 2000$$

$$20 \times 50 = 1000$$

```
train_generator =  
train_datagen.flow_from_directory(  
    train_dir,  
    target_size=(150, 150),  
    batch_size=20,  
    class_mode='binary')  
  
validation_generator =  
test_datagen.flow_from_directory(  
    validation_dir,  
    target_size=(150, 150),  
    batch_size=20,  
    class_mode='binary')
```

- Training: 2,000 images
- Validation: 1,000 images
- Test: 1,000 images

Saving the model

```
model.save('cats_and_dogs_small_1.h5')
```

Training Curves

```
import matplotlib.pyplot as plt

acc = history.history['accuracy']
val_acc = history.history['val_accuracy']
loss = history.history['loss']
val_loss = history.history['val_loss']

epochs = range(1, len(acc) + 1)

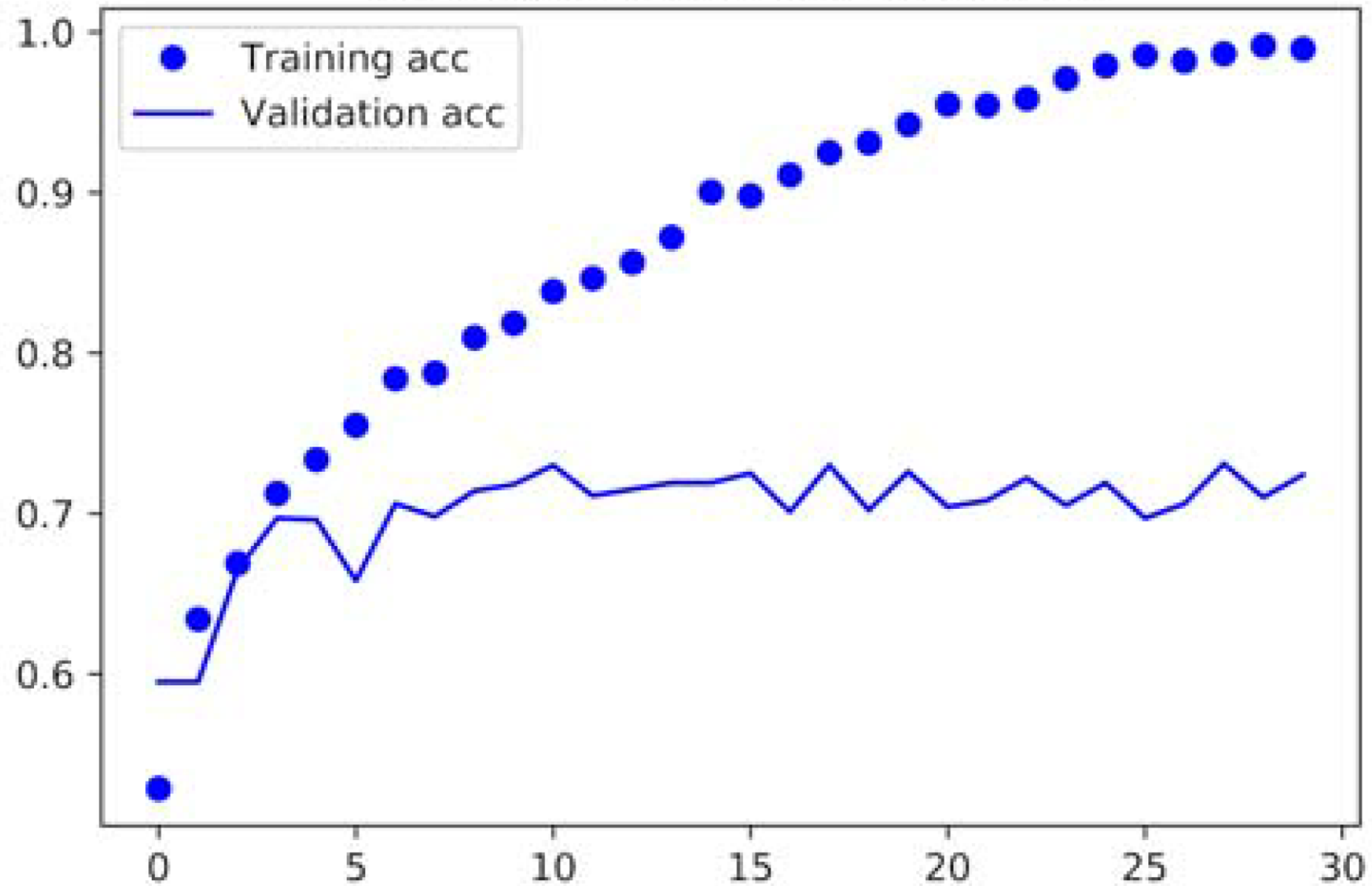
plt.plot(epochs, acc, 'bo', label='Training acc')
plt.plot(epochs, val_acc, 'b', label='Validation acc')
plt.title('Training and validation accuracy')
plt.legend()

plt.figure()

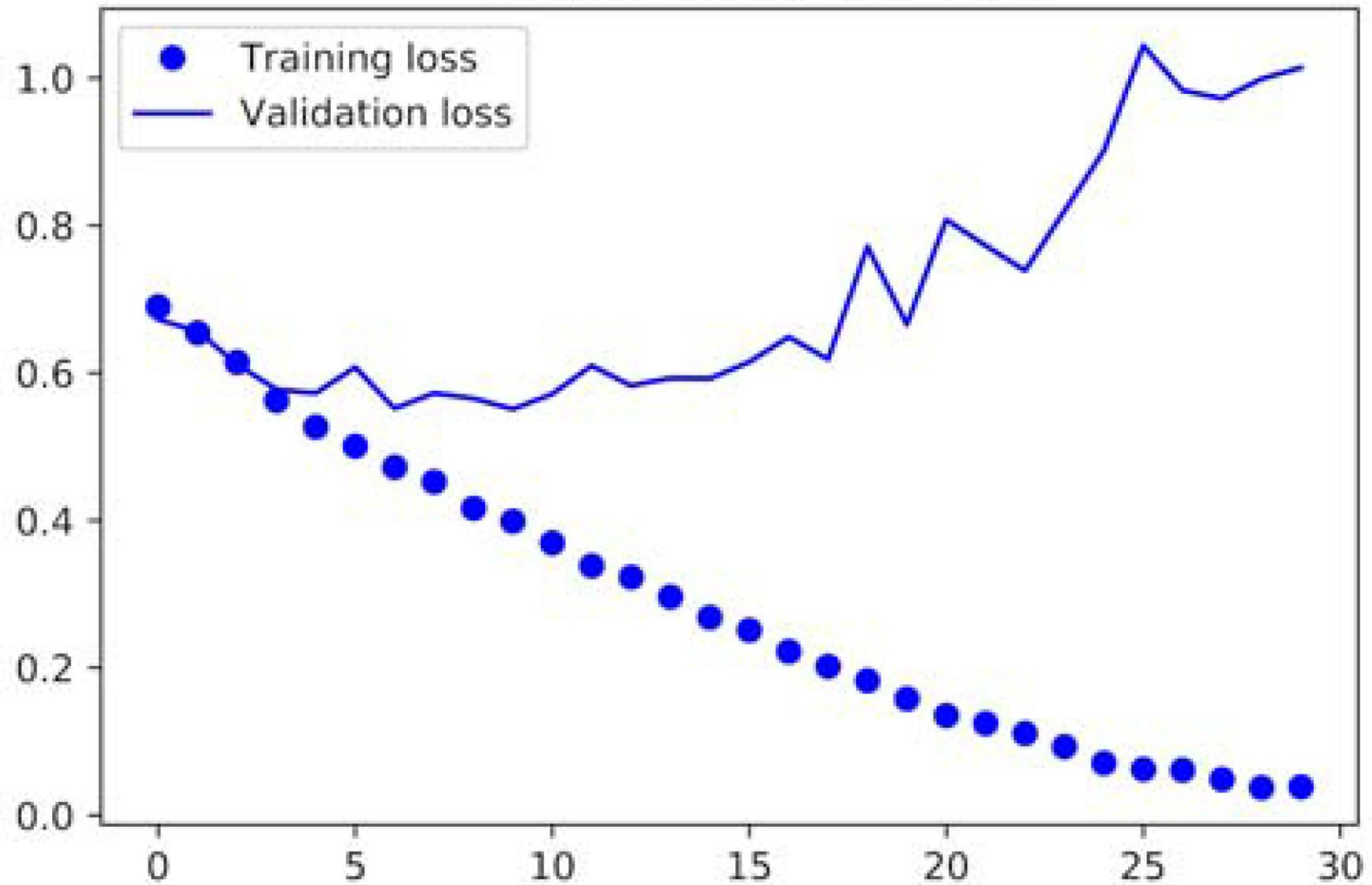
plt.plot(epochs, loss, 'bo', label='Training loss')
plt.plot(epochs, val_loss, 'b', label='Validation loss')
plt.title('Training and validation loss')
plt.legend()

plt.show()
```

Training and validation accuracy



Training and validation loss



Training Curves

Training vs validation

- Training accuracy $\uparrow$ เกือบ 100%
- Validation accuracy $\approx$ 70–72%
- Validation loss เพิ่มขึ้นหลัง epoch แรก ๆ

Data Augmentation

Data Augmentation

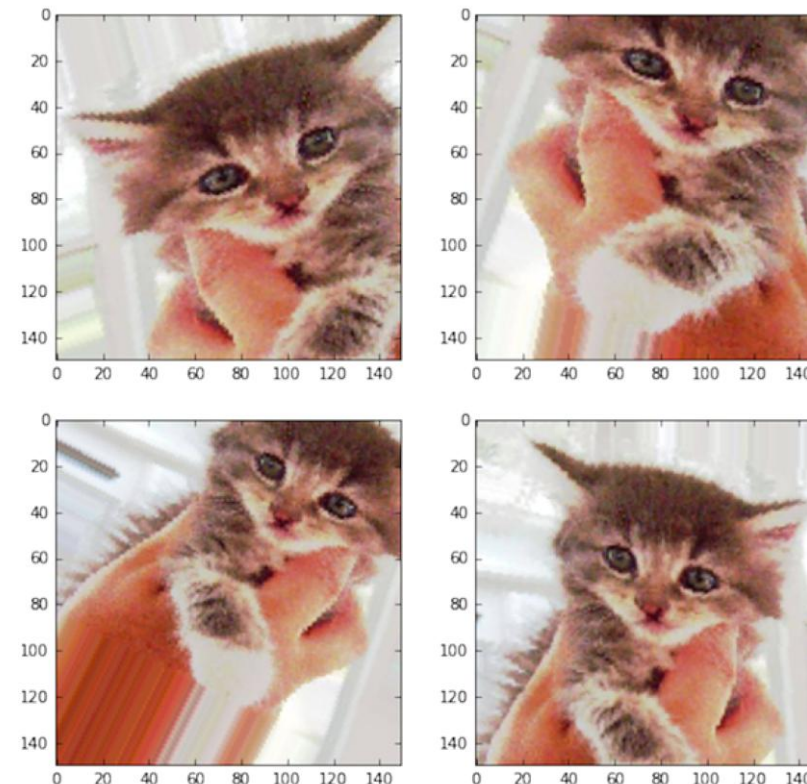
- ปัญหาหลักของ dataset ขนาดเล็ก → Overfitting
- โมเดลจำ training data ได้ดีแต่ generalize ไม่ได้
- แนวคิด:
เพิ่ม “ความหลากหลายของข้อมูล” โดยไม่ต้องเก็บข้อมูลใหม่

Concept of Data Augmentation

สร้าง training samples ใหม่ จาก existing images ใช้ random transformations

ภาพที่ได้:

- ดูสมจริง (believable)
- label ไม่เปลี่ยน
- ระหว่าง training โมเดลแทบไม่เห็นภาพเดิมซ้ำ



ImageDataGenerator in Keras

ImageDataGenerator (Keras)

- เครื่องมือสำหรับทำ **data augmentation**

การทำงาน:

- อ่านภาพ
- แปลงแบบสุ่ม
- ส่งเข้า **model** ระหว่าง **training**

ไม่เก็บภาพใหม่ลง **disk** แต่ใช้การสร้างแบบ **on-the-fly**

ImageDataGenerator in Keras

```
datagen = ImageDataGenerator(  
    rotation_range=40,  
    width_shift_range=0.2,  
    height_shift_range=0.2,  
    shear_range=0.2,  
    zoom_range=0.2,  
    horizontal_flip=True,  
    fill_mode='nearest')
```

Meaning of Each Parameter

Key parameters

- `rotation_range`: หมุนภาพ (degrees)
- `width_shift_range`, `height_shift_range`: เลื่อนภาพ (fraction)
- `shear_range`: shear transform
- `zoom_range`: zoom in/out
- `horizontal_flip`: กลับภาพแนวนอน
- `fill_mode`: เติม pixel ที่หายไป

```
from keras.preprocessing import image
```

```
# ดึงชื่อไฟล์ภาพทั้งหมดในโฟลเดอร์
```

```
fnames = [os.path.join(train_cats_dir, fname) for fname in  
os.listdir(train_cats_dir)]
```

```
# เลือกภาพ 1 รูป
```

```
img_path = fnames[3]
```

```
# โหลดภาพ และ resize เป็น 150x150
```

```
img = image.load_img(img_path, target_size=(150, 150))
```

```
# แปลงเป็น NumPy array (150, 150, 3)
```

```
x = image.img_to_array(img)
```

```
# reshape เป็น (1, 150, 150, 3) เพราะ datagen ทำงานกับ batch
```

```
x = x.reshape((1,) + x.shape)
```

```
# สร้างภาพ augmented
i = 0
for batch in datagen.flow(x, batch_size=1):
    plt.figure(i)
    imgplot = plt.imshow(image.array_to_img(batch[0]))
    i += 1
    if i % 4 == 0:
        break

plt.show()
```

Visualizing Augmented Images

visualize augmentation เพื่อ

1. ตรวจสอบว่า:

- ภาพยังสมเหตุสมผล
- label ยังถูกต้อง

2. ป้องกัน:

- augment แรงเกินไป
- ทำลายความหมายของภาพ (semantic)



Limitation of Data Augmentation

“you can’t produce new information”

Important limitation

ภาพที่ได้ยังมาจาก: ภาพเดิม

ข้อมูลใหม่จากการทำ augmentation: ไม่ได้เพิ่ม information ใหม่ แค่ remix information เดิม

Overfitting:

- ลดลง
- แต่ไม่หาย 100%

Adding Dropout

Dropout for further regularization

- วางก่อน **classifier** (ก่อน **dense**)
- ปิด **neuron** บางตัว ปิดแบบสุ่ม
- บังคับให้ **model**:
 - ไม่พึ่ง **feature** ใด **feature** หนึ่งมากเกินไป

```
from tensorflow.keras import optimizers
model = models.Sequential()
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(150,
150, 3)))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(128, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(128, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))

model.add(layers.Flatten())
model.add(layers.Dropout(0.5))
model.add(layers.Dense(512, activation='relu'))
model.add(layers.Dense(1, activation='sigmoid'))

model.compile(loss='binary_crossentropy',
              optimizer=optimizers.RMSprop(learning_rate=1e-4),
              metrics=['accuracy'])
```

Training with Data Augmentation

Training setup

- training data:
 - rescale + augmentation

validation data:

- rescale only

validation data ห้าม augment

```
train_datagen = ImageDataGenerator(  
    rescale=1./255,  
    rotation_range=40,  
    width_shift_range=0.2,  
    height_shift_range=0.2,  
    shear_range=0.2,  
    zoom_range=0.2,  
    horizontal_flip=True)
```

```
test_datagen = ImageDataGenerator(rescale=1./255)
```

```
train_generator = train_datagen.flow_from_directory(  
    train_dir,  
    target_size=(150, 150),  
    batch_size=32,  
    class_mode='binary')
```

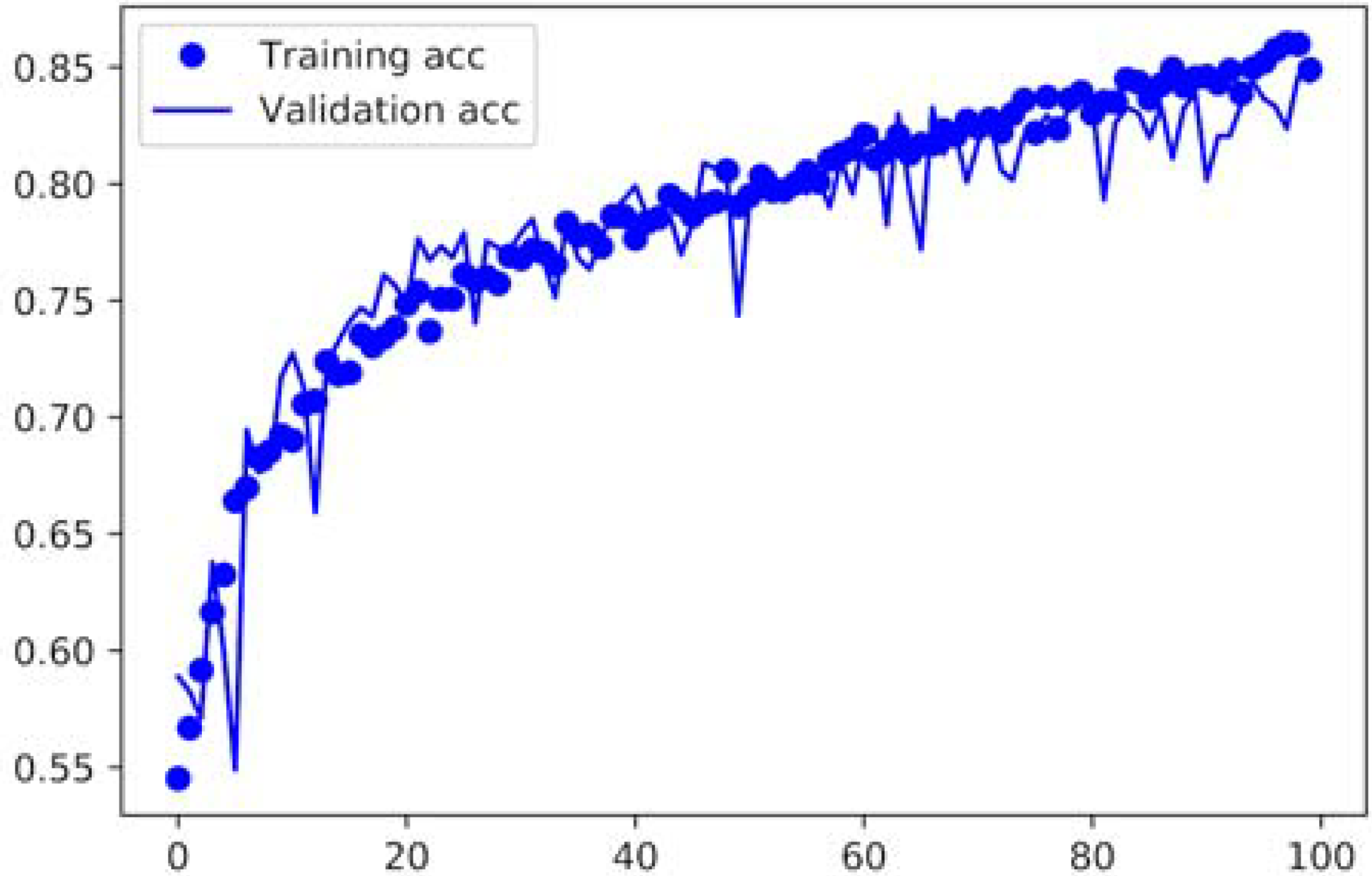
```
validation_generator = test_datagen.flow_from_directory(  
    validation_dir,  
    target_size=(150, 150),  
    batch_size=32,  
    class_mode='binary')
```

```
history = model.fit(  
    train_generator,  
    steps_per_epoch=100,  
    epochs=100,  
    validation_data=validation_generator,  
    validation_steps=50)
```

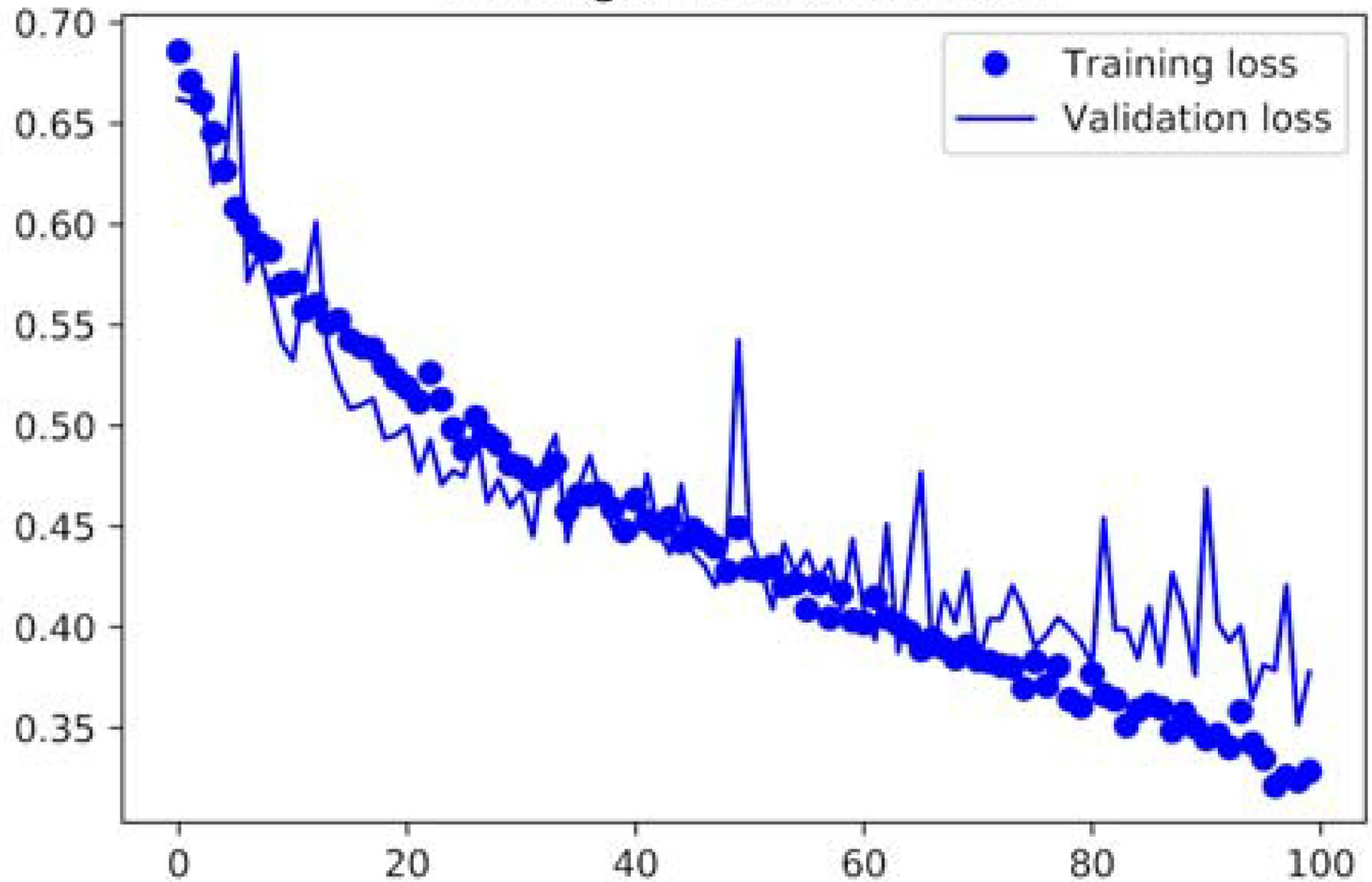
```
model.save('cats_and_dogs_small_2.h5')
```

```
import matplotlib.pyplot as plt
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']
loss = history.history['loss']
val_loss = history.history['val_loss']
epochs = range(1, len(acc) + 1)
plt.plot(epochs, acc, 'bo', label='Training acc')
plt.plot(epochs, val_acc, 'b', label='Validation acc')
plt.title('Training and validation accuracy')
plt.legend()
plt.figure()
plt.plot(epochs, loss, 'bo', label='Training loss')
plt.plot(epochs, val_loss, 'b', label='Validation loss')
plt.title('Training and validation loss')
plt.legend()
plt.show()
```

Training and validation accuracy



Training and validation loss



Training Results

Result

Training & validation curves:

- โกล์กันมาก

Overfitting:

- ลดลงชัดเจน

Accuracy:

- ~82%
- +15% จาก model เดิม

Pretrained convnet

Using a Pretrained ConvNet

- Image dataset ขนาดเล็ก
- train model เริ่มต้นการเรียนรู้จาก ศูนย์ 0
- Pretrained ConvNet ใช้ความรู้จาก model ที่ train มาแล้ว
- แนวคิด: reuse learned features

What is a Pretrained Network?

- Network ที่ train บน dataset ขนาดใหญ่ (เช่น ImageNet)
- เรียนรู้ feature ทั่วไปของภาพ
- ใช้ซ้ำกับปัญหาใหม่ได้ (transferability)

Why It Works for Small Datasets

- Feature learned = general visual knowledge
- ไม่ต้องเรียนรู้ใหม่ทั้งหมด
- เหมาะมากกับ small-data problems

Example: ImageNet → Cats vs Dogs

- ImageNet มี class สัตว์จำนวนมาก
- มีหลายสายพันธุ์ของ cats และ dogs
- เหมาะกับ dogs-vs-cats classification

Model Used: VGG16

- ConvNet architecture จากปี 2014 เก่า แต่ยังดีอยู่
- Simple, เข้าใจง่าย
- ใช้กันแพร่หลายใน ImageNet

Two Ways to Use a Pretrained Network

- Feature Extraction
- Fine-Tuning

Feature Extraction with Pretrained ConvNet

Feature Extraction with Pretrained ConvNet

Feature Extraction (การสกัดคุณลักษณะ)

- ใช้ pretrained weights ของ model ที่ train มาแล้ว
- สกัด feature จาก input ใหม่
- train classifier ใหม่ โดยใช้ weights ที่ initialize ใหม่

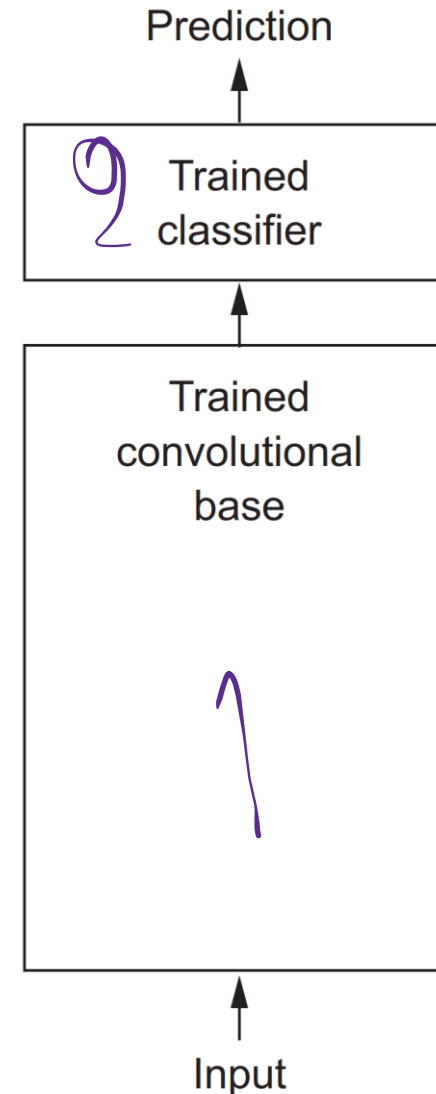
โครงสร้าง ConvNet สำหรับ Image Classification

1) Convolutional base

- Conv + Pooling layers
- ใช้เป็น pretrained weights

2) Densely connected classifier

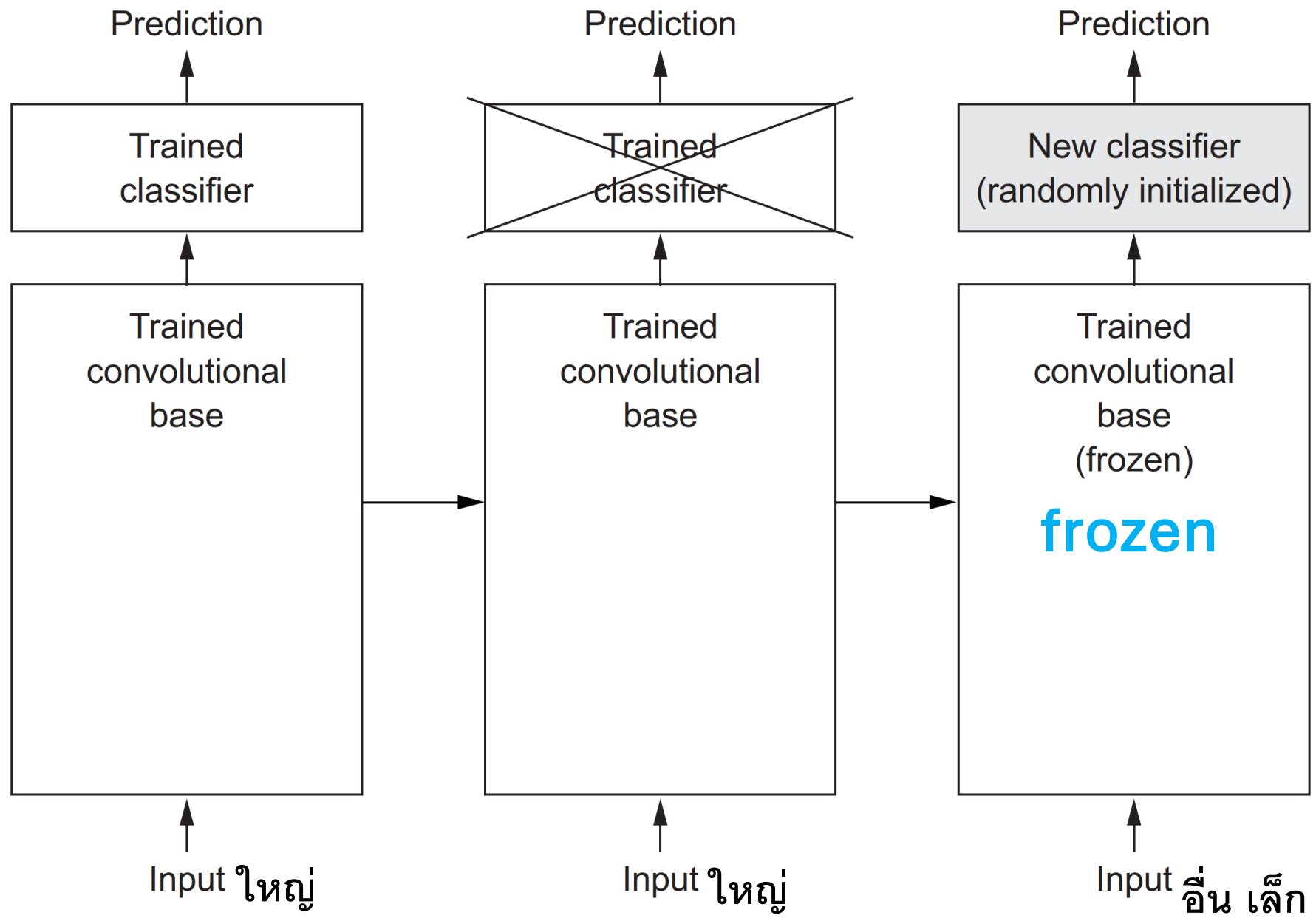
- Dense layers
- ใช้ weights ใหม่



Concept of Feature Extraction

Input image

- Pretrained convolutional base (frozen weights)
- Feature maps
- New classifier (trainable weights)
- Prediction



Reusing the Convolutional Base ?

reuse เฉพาะ convolutional base เนื่องจาก

- weights มีความเป็น generic ใช้ได้กับหลาย vision tasks
- classifier (dense) มี weights เฉพาะกับ task เดิม ทำให้ไม่เหมาะที่จะทำใช้กับ task เฉพาะ

Dense classifier ไม่เหมาะ reuse

ข้อจำกัดของ Dense classifier

- weights specific ต่อ class set เดิม
- ไม่เก็บ spatial information
- ไม่ generalize ไป task ใหม่ได้ดี

Depth ของ layer กับลักษณะของ weights

Early layers

- weights → edges, colors, textures

Deeper layers

- weights → object parts, concepts

Using Pretrained VGG16

Pretrained ConvNet: VGG16

- Pretrained weights จาก ImageNet
- ใช้เป็น convolutional base
- ไม่ใช้ classifier เดิม

conv_base

```
conv_base = VGG16(  
    weights='imagenet',  
    include_top=False,  
    input_shape=(150, 150, 3)  
)  
Conv_base.summary( )
```

Output ของ convolutional base

```
>>> conv_base.summary()
```

Layer (type)	Output Shape	Param #			
=====					
input_1 (InputLayer)	(None, 150, 150, 3)	0	block4_conv1 (Convolution2D)	(None, 18, 18, 512)	1180160
block1_conv1 (Convolution2D)	(None, 150, 150, 64)	1792	block4_conv2 (Convolution2D)	(None, 18, 18, 512)	2359808
block1_conv2 (Convolution2D)	(None, 150, 150, 64)	36928	block4_conv3 (Convolution2D)	(None, 18, 18, 512)	2359808
block1_pool (MaxPooling2D)	(None, 75, 75, 64)	0	block4_pool (MaxPooling2D)	(None, 9, 9, 512)	0
block2_conv1 (Convolution2D)	(None, 75, 75, 128)	73856	block5_conv1 (Convolution2D)	(None, 9, 9, 512)	2359808
block2_conv2 (Convolution2D)	(None, 75, 75, 128)	147584	block5_conv2 (Convolution2D)	(None, 9, 9, 512)	2359808
block2_pool (MaxPooling2D)	(None, 37, 37, 128)	0	block5_conv3 (Convolution2D)	(None, 9, 9, 512)	2359808
block3_conv1 (Convolution2D)	(None, 37, 37, 256)	295168	block5_pool (MaxPooling2D)	(None, 4, 4, 512)	0
block3_conv2 (Convolution2D)	(None, 37, 37, 256)	590080	=====		
block3_conv3 (Convolution2D)	(None, 37, 37, 256)	590080	Total params: 14,714,688		
block3_pool (MaxPooling2D)	(None, 18, 18, 256)	0	Trainable params: 14,714,688		
			Non-trainable params: 0		

สรุป

Feature Extraction

- ไม่ได้ **train** โมเดลจากศูนย์ทั้งหมด
- Reuse pretrained weights (conv base)
- Train เฉพาะ weights ของ classifier ใหม่

วิธีทำ Feature Extraction

Feature Extraction มี 2 วิธีหลักๆ คือ

Strategy 1:

- Precompute features
- Train classifier แยกต่างหาก

Strategy 2:

- ต่อ classifier บน conv_base
- Train แบบ end-to-end
- Freeze convolutional base

Strategy 1 — Precompute Features

Strategy 1: Precompute Features

Input image

→ conv_base (pretrained weights)

→ Save feature maps

→ Train Dense classifier

```
import os
import numpy as np
from keras.preprocessing.image import ImageDataGenerator

base_dir = '/Users/fchollet/Downloads/cats_and_dogs_small'
train_dir = os.path.join(base_dir, 'train')
validation_dir = os.path.join(base_dir, 'validation')
test_dir = os.path.join(base_dir, 'test')

datagen = ImageDataGenerator(rescale=1./255)
batch_size = 20
```



แก้ไข

```
def extract_features(directory, sample_count):
    features = np.zeros(shape=(sample_count, 4, 4, 512))
    labels = np.zeros(shape=(sample_count))
    generator = datagen.flow_from_directory(
        directory,
        target_size=(150, 150),
        batch_size=batch_size,
        class_mode='binary')
    i = 0
    for inputs_batch, labels_batch in generator:
        features_batch = conv_base.predict(inputs_batch)
        features[i * batch_size : (i + 1) * batch_size] = features_batch
        labels[i * batch_size : (i + 1) * batch_size] = labels_batch
        i += 1
        if i * batch_size >= sample_count:
            break
    return features, labels
```

```
train_features, train_labels = extract_features(train_dir, 2000)
validation_features, validation_labels = extract_features(validation_dir, 1000)
test_features, test_labels = extract_features(test_dir, 1000)
```

Strategy 1: Precompute Features

Input image

→ conv_base (pretrained weights)

→ Save feature maps

→ Train Dense classifier

Flatten ก่อนเข้า Dense

```
train_features = np.reshape(train_features, (2000, 4 * 4 * 512))
```

```
validation_features = np.reshape(validation_features, (1000, 4 * 4 * 512))
```

```
test_features = np.reshape(test_features, (1000, 4 * 4 * 512))
```

Train Dense Classifier

```
from keras import models
from keras import layers
from keras import optimizers

model = models.Sequential()
model.add(layers.Dense(256, activation='relu', input_dim = 4 * 4 * 512))
model.add(layers.Dropout(0.5))
model.add(layers.Dense(1, activation='sigmoid'))

model.compile(optimizer=optimizers.RMSprop(lr=2e-5),
              loss='binary_crossentropy',
              metrics=['acc'])

history = model.fit(train_features, train_labels,
                    epochs=30,
                    batch_size=20,
                    validation_data=(validation_features, validation_labels))
```

ผลของ Strategy 1

```
import matplotlib.pyplot as plt
```

```
acc = history.history['acc']
```

```
val_acc = history.history['val_acc']
```

```
loss = history.history['loss']
```

```
val_loss = history.history['val_loss']
```

```
epochs = range(1, len(acc) + 1)
```

```
plt.plot(epochs, acc, 'bo', label='Training acc')
```

```
plt.plot(epochs, val_acc, 'b', label='Validation acc')
```

```
plt.title('Training and validation accuracy')
```

```
plt.legend()
```

```
plt.figure()
```

```
plt.plot(epochs, loss, 'bo', label='Training loss')
```

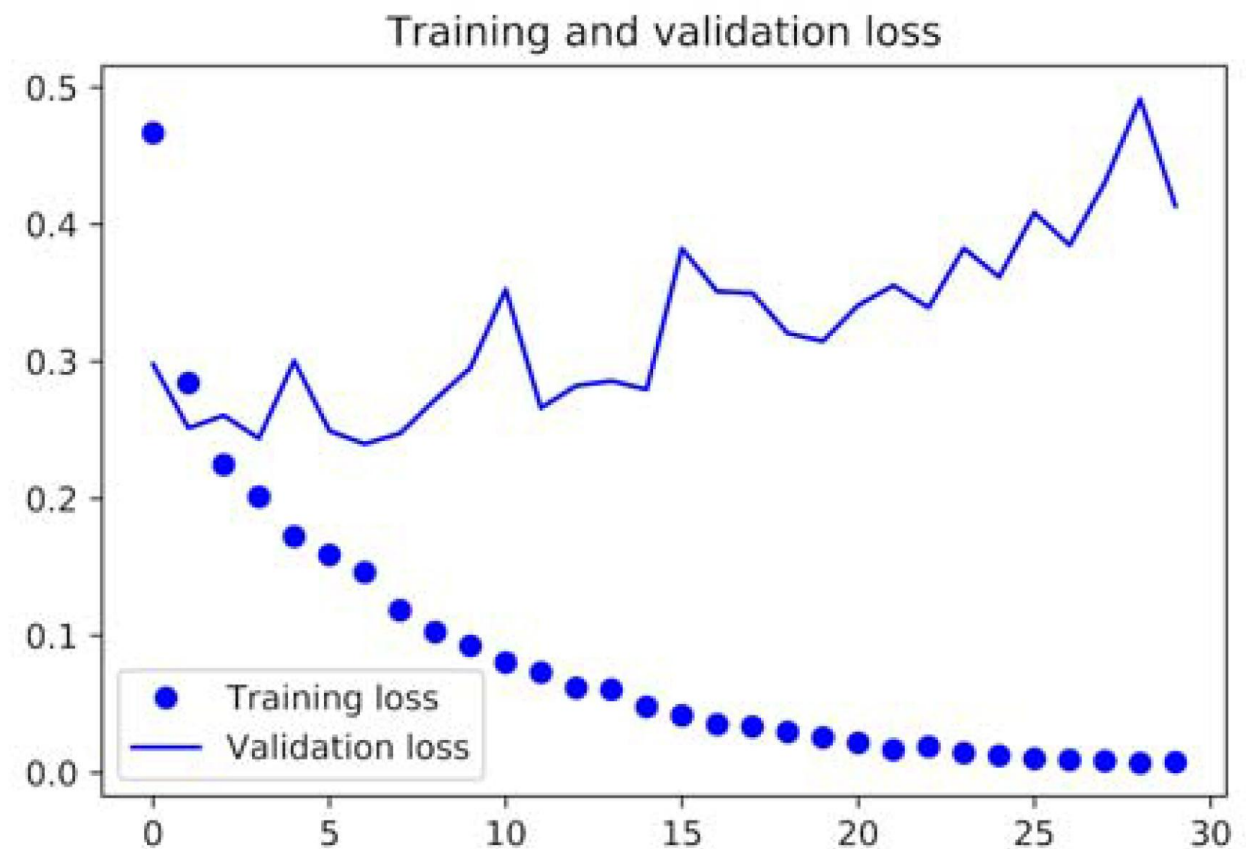
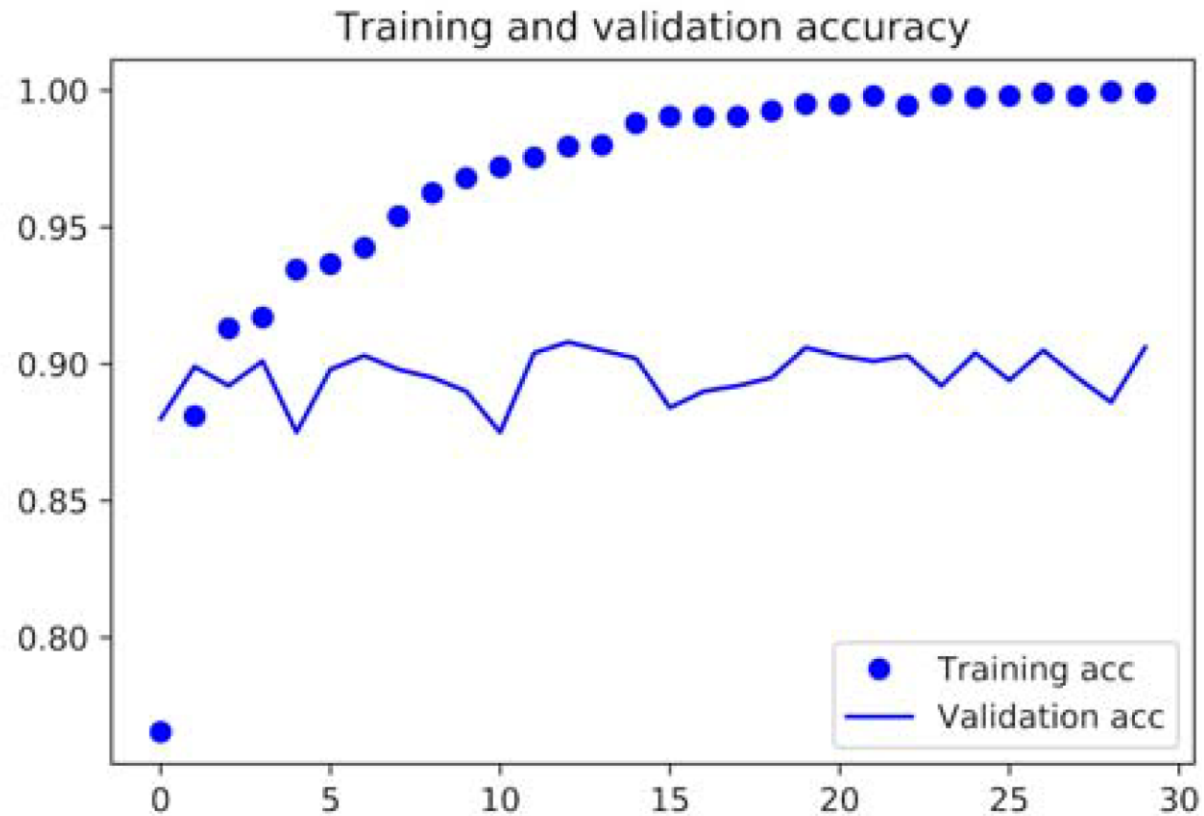
```
plt.plot(epochs, val_loss, 'b', label='Validation  
loss')
```

```
plt.title('Training and validation loss')
```

```
plt.legend()
```

```
plt.show()
```

ผลของ Strategy 1



Strategy 2 End-to-End Training (Frozen conv_base)

Input image

- Data Augmentation (train only)
- conv_base (pretrained weights, frozen)
- Feature maps
- Flatten
- Dense classifier (trainable weights)
- Prediction

code

```
from keras import models
from keras import layers

model = models.Sequential()
model.add(conv_base)
model.add(layers.Flatten())
model.add(layers.Dense(256, activation='relu'))
model.add(layers.Dense(1, activation='sigmoid'))

model.summary()
```

model

Freeze weight

Layer (type)	Output Shape	Param #
=====		
vgg16 (Model)	(None, 4, 4, 512)	14714688
flatten_1 (Flatten)	(None, 8192)	0
dense_1 (Dense)	(None, 256)	2097408
dense_2 (Dense)	(None, 1)	257
=====		

Total params: 16,812,353
Trainable params: 16,812,353
Non-trainable params: 0

Freeze weight

```
>>> print('This is the number of trainable weights '
...       'before freezing the conv base:', len(model.trainable_weights))
This is the number of trainable weights before freezing the conv base: 30
```

```
>>> conv_base.trainable = False
```

```
>>> print('This is the number of trainable weights '
...       'after freezing the conv base:', len(model.trainable_weights))
This is the number of trainable weights after freezing the conv base: 4
```

Data Augmentation

```
ImageDataGenerator(  
    rotation_range=40,  
    width_shift_range=0.2,  
    zoom_range=0.2,  
    horizontal_flip=True  
)
```

code

```
from keras.preprocessing.image import ImageDataGenerator
from keras import optimizers
```

AUGMENTATION

```
train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=40,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest')
```

```
test_datagen = ImageDataGenerator(rescale=1./255)
```

```
train_generator = train_datagen.flow_from_directory(  
    train_dir,  
    target_size=(150, 150),  
    batch_size=20,  
    class_mode='binary')
```

```
validation_generator = test_datagen.flow_from_directory(  
    validation_dir,  
    target_size=(150, 150),  
    batch_size=20,  
    class_mode='binary')
```

```
model.compile(loss='binary_crossentropy',  
              optimizer=optimizers.RMSprop(lr=2e-5),  
              metrics=['acc'])
```

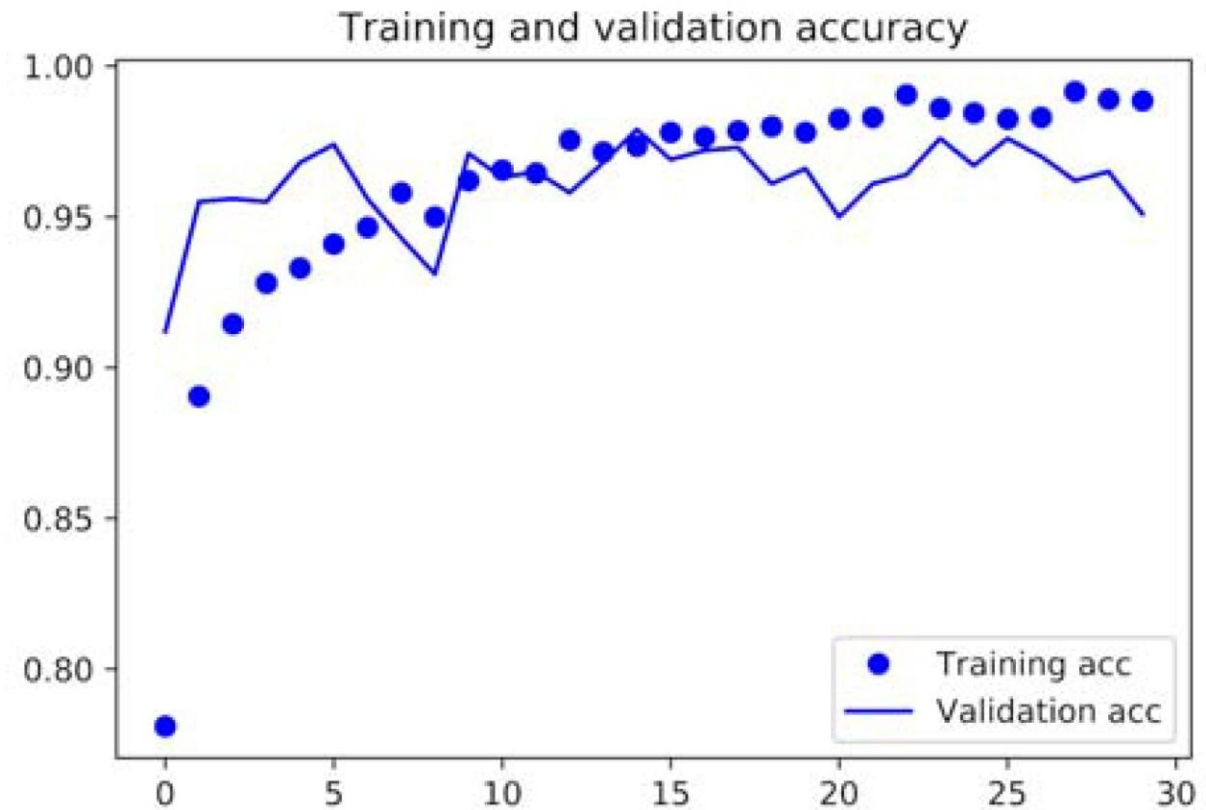
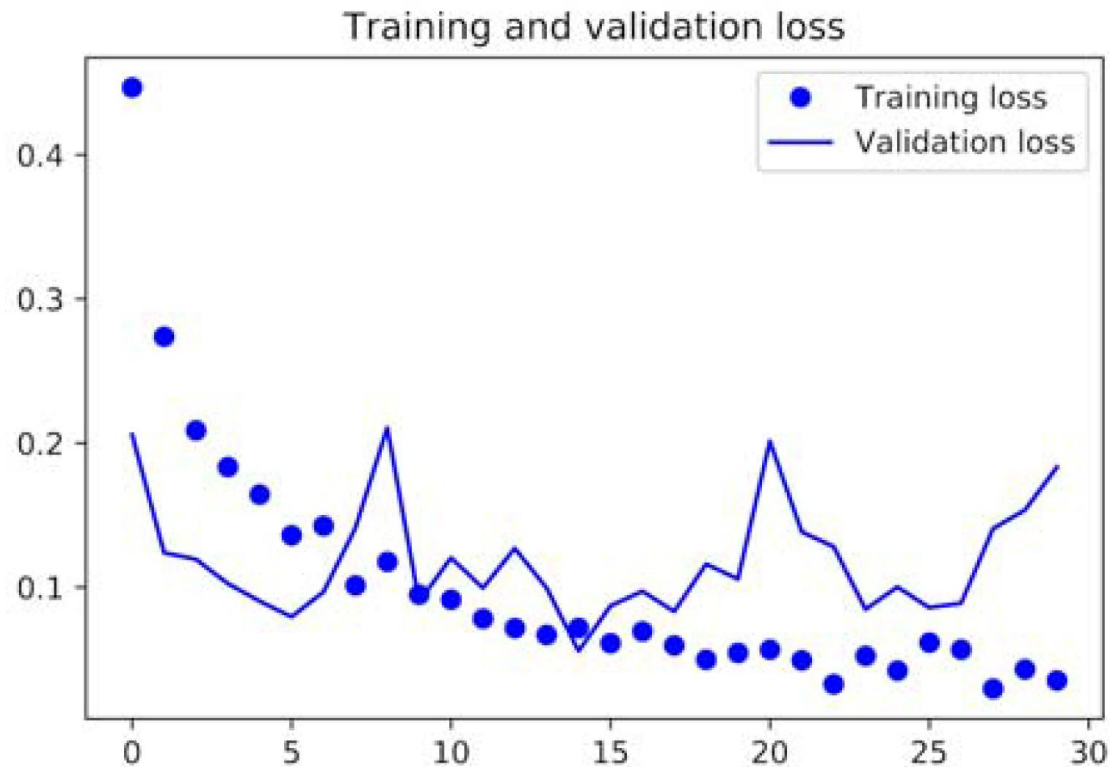
```
history = model.fit_generator(  
    train_generator,  
    steps_per_epoch=100,  
    epochs=30,  
    validation_data=validation_generator,  
    validation_steps=50)
```

ผล Strategy 2

Validation accuracy $\approx 96\%$

Generalization ดีขึ้น

Overfitting ลดลง



เทียบ 2 วิธี

Strategy 1

- เร็ว ใช้ CPU ได้
- ไม่มี augmentation

Strategy 2 (End-to-End)

- ช้า ต้องใช้ GPU
- ใช้ augmentation ได้
- Performance ดีกว่า

lab

- ใช้ทำ แสดงภาพ **cnn**
- ถ้ามโครงสร้างของ **Cnn** ให้อภิปรายในชั้น
- ถ้าม **feature** ต่างๆของ **augmentation**